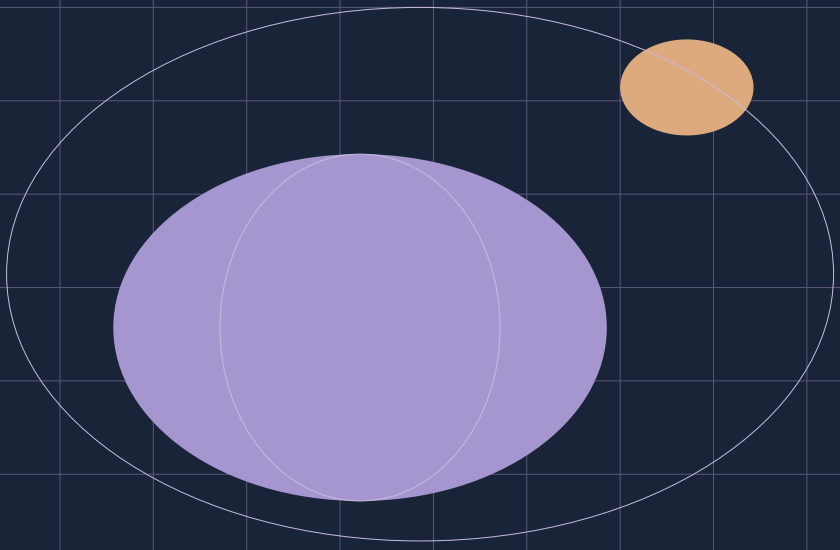
BINARY ASTEROID SCIENCE

BASCO

The user manual

Build a study. Follow the mathematics.
Understand the uncertainty.



A practical guide and API companion

Hou gravity and AD/native derivatives; masses and gravity fields; observations, priors and rank; Hera reproduction; shape ensembles; multi-arc covariance, reports, examples and extension contracts.

BASCO 0.1.0 / Documentation edition 2026-10-19
Scientific source: c00d5ff / linked contents and examples

Complete searchable reference: ashcav1.github.io/BASCO-docs/

REFERENCE

Printable handbook

A continuous reading edition, with linked chapters and a public API map.

BASCO 0.1.0 Hou AD default · native available Source c00d5ff

This reading edition combines the authored guides and public API map. The website holds the complete searchable reference, including internal and legacy entries. Use your browser's Save as PDF for a fresh copy of this page.

Contents

1. Your first covariance study
2. What BASCO computes
3. Compose a study from scratch
4. Bodies, moments, units and frames
5. Gravity order and estimated parameters
6. Observations, barycenter and schedules
7. Noise, priors and numerical support
8. Reproduce the Hera paper
9. Results, plots and exported files
10. Reuse and trade studies
11. Shape ensembles and Kaula priors
12. Independent arcs and estimation
13. AD, analytic derivatives and validation
14. Extend one component at a time
15. Example gallery
16. Capabilities and limits
17. Troubleshooting and reproducibility
18. Glossary and notation
19. Public API map

CHAPTER 01

Your first covariance study

BASCO studies how observations constrain the masses, inertias and gravity fields of a binary asteroid. An ordinary script supplies the bodies, nominal state, gravity fidelity, estimated quantities and measurement assumptions. The library owns propagation, derivatives, row assembly, uncertainty analysis and consistent reporting.

Start with the short script below. It exercises the same public components as the Hera reproduction. Its one-hour duration deliberately leaves unresolved gravity directions; an honest rank-deficient answer is a useful first result.

Prepare the repository environment

Run these commands from the BASCO checkout, using a host Python 3.11 or newer and `uv`:

```
python3 -B tools/dev.py bootstrap
python3 -B tools/dev.py build
python3 -B tools/dev.py ad_environment
python3 -B tools/dev.py test
```

The build requires a C++ compiler, CMake, pkg-config, Armadillo and OpenBLAS development libraries. The managed helper stores its interpreter, installed wheel, native build and command logs in the machine-local BASCO cache. Scientific scripts run against that installed package outside the checkout. After editing library source, rebuild; the installed wheel does not follow source edits automatically.

The currently validated platform is macOS arm64. The native dependencies on the validated machine require macOS 26.0. A successful run on another platform remains a separate portability check.

If installing a built package instead of using the repository helper, request the recommended optional dependencies as `basco[ad,plot]`. BASCO still needs its compiled native extensions. The runtime troubleshooting guide distinguishes host, build and scientific environments.

A complete script

Save this as `my_binary.py` in the repository. Every physical number below uses metres, kilograms, seconds and radians. `axes` are semi-axes. This is the same composition validated in the existing getting-started guide.

```python
from pathlib import Path

import numpy as np
import basco as bc

primary = bc.Body(
    "primary",
    bc.UniformEllipsoid(axes=[400.0, 350.0, 300.0], density=2200.0),
)
secondary = bc.Body(
    "secondary",
    bc.UniformEllipsoid(axes=[90.0, 75.0, 60.0], density=2200.0),
)

G = bc.G_SI
separation = 2000.0
n = np.sqrt(G * (primary.mass + secondary.mass) / separation**3)
state = bc.BinaryState(
    position=[separation, 0.0, 0.0],
    velocity=[0.0, separation * n, 0.0],
    omega_primary=[0.0, 0.0, 2.0 * np.pi / 14400.0],
    omega_secondary=[0.2 * n, 0.1 * n, n],
    A_to_N=np.eye(3),
    B_to_A=np.eye(3),
)
system = bc.BinarySystem(primary, secondary, state, G=G, name="my binary")
model = bc.BinaryDynamics(system, gravity=bc.HouGravity(mutual_order=6))

parameters = (
    bc.Parameters(model)
    .estimate_masses()
    .estimate_moments(degrees=(2,))
    .estimate_harmonics(degrees=(3, 4))
    .estimate_initial_state()
)
measurements = bc.ObservationPlan.uniform(
    stop=3600.0,
    cadence=600.0,
    channels=(
        bc.RelativePosition(sigma=1.0),
        bc.BodyAttitude("primary", sigma=np.deg2rad(1.0)),
        bc.BodyAttitude("secondary", sigma=np.deg2rad(1.0)),
        bc.PrimaryBarycentricPosition(
            sigma=1.0,
            nuisance=bc.PolynomialBias("barycenter", degree=1),
        ),
    ),
)
study = bc.CovarianceStudy(
    model,
    parameters,
    measurements,
    integrator=bc.RK4(dt=60.0),
    priors=(),
    solver=bc.SquareRootSolver(),
)

print(study.describe()["unknown_labels"])
rows = study.linearize()
result = rows.covariance()
print(result.summary())
print(result.mass_properties().summary())

folder = bc.export_study(
    rows,
```

```
    result,
    root=Path(__file__).resolve().parent / "results",
    label="my-binary-g06-e04",
)
print(folder)
```

Run it with:

```
python3 -B tools/dev.py run my_binary.py
```

This chooses Hou automatic differentiation. JAX is optional at installation/import time, but required when this default provider propagates. Missing JAX produces an actionable error; BASCO never silently chooses a different derivative method.

The launcher starts outside the repository. Script-relative paths should use `Path(__file__)`, as the example does. The returned export path points to tables, diagnostics, named arrays and a checksum manifest. See reading results before interpreting uncertainty numbers.

Choose native analytic derivatives explicitly

Add this keyword to the study constructor:

```
transport=bc.NativeAnalyticTransport(),
```

Then run:

```
python3 -B tools/dev.py run --native my_binary.py
```

Both choices matter: the constructor chooses the scientific provider, while `--native` chooses an environment without JAX. The native route preserves the hand-derived Hou partials and the same physical model and parameter map. Derivative methods explains what is shared and what is independently checked.

Add figures

Prepare and use the plotting environment:

```
python3 -B tools/dev.py plot_environment
python3 -B tools/dev.py run --plot my_binary.py
```

Inside the script, add `figures=True, theme=bc.PlotTheme(tex=False)` to `export_study`. Math text works without a TeX installation. `tex=True` explicitly requests external LaTeX. The shared paper-inspired palette and theme apply across the standard plots.

Where to go next

GOAL	READ NEXT
Understand every component in this script	Build a study
Supply your own geometry, frames or state	Bodies and states
Change what is estimated	Parameters
Add data channels, gaps or barycenter uncertainty	Observations
Compare noise, priors or degrees cheaply	Reuse and trades
Inspect signatures and member definitions	Public components

The custom-binary example demonstrates Astropy quantities. The Hera runner is the full paper recipe rather than a short smoke study.

CHAPTER 02

What BASCO computes

BASCO's main question is: **given a nominal binary and an observing plan, how well can the selected observations constrain its masses and gravity field?** It linearizes predictions about that nominal, combines measurement weights and stated prior information, then diagnoses supported uncertainty directions.

A covariance forecast is local to the supplied model, state, observation geometry and statistical assumptions. It does not by itself fit actual data, certify the nominal truth or measure force-model truncation bias.

The computation in six stages

STAGE	INPUT	OUTPUT	MAIN OWNER
Physical model	Bodies, moments, relative state and mutual gravity order	Nominal full two-body dynamics	BinaryDynamics
Solve-for map	Selected masses, inertia/field coordinates and initial-state treatment	Named independent coordinates and raw-moment Jacobian	Parameters
Variational propagation	Model, coordinates, integration grid and derivative provider	States, state transition and parameter sensitivities	Transport
Observation assembly	Propagation, channels, direct derivatives and nuisance	Physical unweighted design rows	LinearizedStudy
Statistical solve	Rows, noise, priors, scales and numerical support policy	Covariance, nullspace and diagnostics	CovarianceResult
Reporting	Result plus physical maps	Mass/gravity tables, plots and verified exports	make_presentation

The same components serve a small user script and the Hera example. Noise changes do not belong in the gravity kernel; a plotting change does not belong in a covariance solver.

Variational equations

Let `x` denote the native integration state and `p` the independent physical parameters. The dynamics and local derivatives give:

$$\dot{x} = F(x, p), \qquad A = \frac{\partial F}{\partial x}, \qquad B = \frac{\partial F}{\partial p}.$$

The propagated state transition and sensitivities satisfy:

$$\dot{\Phi} = A\Phi, \quad \Phi(0) = I, \qquad \dot{\Psi} = A\Psi + B, \quad \Psi(0) = 0.$$

BASCO transports raw-moment sensitivities and maps them into the independent coordinate basis. A mass direction changes all retained dimensional moments associated with that body's mass. Treating mass as only a monopole derivative would omit part of the implemented physical chain.

For an observation prediction `h`, the design rows include both propagated and direct terms:

$$H_p = h_x \Psi + h_p, \qquad H_{x_0} = h_x \Phi L_0.$$

`L_0` lifts an 18-dimensional physical initial-state error into the 30-component native state. Local observation nuisance supplies additional columns. The direct `h_p` term is essential for Didymos position relative to the system barycenter.

Units and attitude are part of the mathematics

The public physical boundary uses SI and radians. The retained native codec uses kilometre-based translation and matrix attitude entries. BASCO performs the conversions and provides physical tangent maps; users should not independently convert public lengths to kilometres.

Two attitude matrices contain 18 scalar entries, but only six physical attitude-error directions. Covariance uses local SO(3) attitude errors instead of treating all matrix entries as independent rotations. See bodies and states.

Statistical interpretation

Whitening transforms the observation residual/design using its declared covariance. Characteristic column scales make differently dimensioned solve-fors numerically comparable. The numerical solve operates on whitened, scaled rows and maps supported uncertainty back to physical units.

For full supported rank, the familiar expression is:

$$P = (H^T R^{-1} H + J_{\text{prior}}^T R_{\text{prior}}^{-1} J_{\text{prior}})^{-1}.$$

The implementation uses square-root QR/SVD methods rather than forming and inverting that normal matrix directly. Rank decisions, eliminated nuisance and physical scaling are documented outputs, not invisible implementation details.

A scale is not a prior. A prior adds knowledge; a numerical threshold selects what the finite-precision calculation supports. A fixed parameter is conditioned on its nominal, not estimated perfectly by the observations.

Two derivative routes, one maintained gravity path

Hou mutual gravity at total orders 2, 4 and 6 is the main scientific path. Default transport uses automatic differentiation of a separate JAX-compatible Hou value implementation. NativeAnalyticTransport preserves manual analytic derivatives. Both feed the same physical-coordinate, observation and statistical machinery.

Finite differences provide an explicit, optional check. They do not run inside normal covariance analysis. Neither AD nor native agreement alone proves all physics, frames and parameter choices correct; independent equations, finite differences, conservation/refinement and reference reproduction serve different purposes.

Cartesian multipole gravity is deprecated compatibility. Mascons remain available for reference and extension use. These retained models do not change the Hou-centered scope of the toolkit.

What is already an estimator?

BASCO includes frozen-linear multi-arc corrections and a generic callback-based nonlinear batch/MAP driver. Binary observation ingestion, complete measured-attitude residuals, seeded mission-style recovery and sequential filters remain unfinished adapters/features. Multi-arc and estimation makes that boundary explicit.

For implementation details, begin with the mathematical contracts, extension contracts and current validation evidence.

CHAPTER 03

Compose a study from scratch

A readable runner should describe scientific choices. It should not need to manipulate native array slots, rebuild gravity derivatives or know plot-specific indices. BASCO's complete quickstart follows that rule; this chapter explains its composition and the useful objects you retain.

Unless stated otherwise, snippets below are fragments using `import basco as bc` and the model components created in the quickstart.

Select physical inputs first

Create two Body objects from an analytic ellipsoid or a checked moment distribution, then a BinaryState. Put them in BinarySystem, including the fixed gravitational constant `G` and a descriptive system name.

```
system = bc.BinarySystem(primary, secondary, state, G=bc.G_SI, name="my binary")
model = bc.BinaryDynamics(system, gravity=bc.HouGravity(mutual_order=6))
```

Mass distribution, reference radii and initial state all affect the physical problem. Bodies and states explains their units, frames and required moment completeness.

Choose independent unknowns

```
parameters = (
    bc.Parameters(model)
    .estimate_masses()
    .estimate_moments(degrees=(2,))
    .estimate_harmonics(degrees=(3, 4))
    .estimate_initial_state()
)
```

This choice differs from the paper's all-raw degree-4 parameterization. It retains degree-2 inertia and uses independent exterior coefficients at degrees 3 and 4. The model still evaluates order-6 gravity, including fixed higher moments. See parameter choices before changing bases.

Describe observations separately

```
plan = bc.ObservationPlan.uniform(
    stop=3600.0,
    cadence=600.0,
    channels=(
        bc.RelativePosition(sigma=1.0),
        bc.BodyAttitude("primary", sigma=0.01),
        bc.BodyAttitude("secondary", sigma=0.01),
    ),
)
```

These are predicted measurement directions with default independent sigmas. Schedules mask observed rows on this grid while propagation continues. Add barycentric position and nuisance explicitly when that is part of the experiment. Observation configuration covers those choices.

Compose numerical and statistical policies

```
study = bc.CovarianceStudy(
    model,
    parameters,
    plan,
    integrator=bc.RK4(dt=60.0),
    priors=(),
    solver=bc.SquareRootSolver(),
)
print(study.describe())
```

`describe()` resolves the configuration before expensive propagation. The fixed integration step must be compatible with the output cadence. The constructor's default transport is Hou AD. To use the preserved manual route, pass `transport=bc.NativeAnalyticTransport()` explicitly and choose the native runtime environment.

Noise, priors and solving explains correlated weights, prior producers and the separate projected-solver policy.

Keep the linearization

```
rows = study.linearize()
result = rows.covariance()
```

This two-step form is useful for nearly every serious study. `study.run()` is a convenience for linearize-then-solve; repeated calls can repeat propagation. Retaining `rows` lets you change noise, priors and covariance solver with no additional dynamics.

OBJECT	IMPORTANT CONTENTS	NORMAL USE
`study`	Model, resolved parameter intent, observation plan, provider and policies	Describe or produce a linearization
`rows.transport`	Epochs, native states, transition/sensitivity arrays and checked identity	Reuse expensive dynamics
`rows`	Unweighted design rows, exact observation order, coordinate layout, scales, nominal and statistics	Reanalyze assumptions
`result`	Data/posterior support, covariance/nullspace, diagnostics and physical reports	Interpret or export uncertainty
`presentation`	Canonical tables/series/statuses with provenance	Render consistent files and figures

The design-row matrix is `rows.design_rows`; its rows follow `rows.row_labels` / `row_units`, and columns follow `rows.layout.labels` / `units`. Read the layout instead of relying on a remembered mass column index.

Report then export

```
print(result.summary())
print(result.mass_properties().summary())
print(result.harmonics(max_degree=4).summary())
```

Inspect support and assumptions before presenting sigmas. A short experiment can validly leave many parameters unresolved. Results and files covers null-aware reports, covariance blocks and standard exports.

A useful runner structure

For a reusable project runner, keep a function such as `make_study(...)` whose arguments expose the scientific alternatives you actually compare. Let it compose public components and return a normal `CovarianceStudy`. Put command-line parsing, output paths and printing in the runner, outside library physics.

The Hera runner demonstrates this structure. Its factory independently exposes case, gravity order, estimated degrees, raw/harmonic basis, higher-degree body selection, prior, step, duration and transport. The degree-trade runner reuses that factory and checked transports.

Common next changes

CHANGE	COMPONENT TO REPLACE	DOES IT REQUIRE NEW PROPAGATION?
Measurement precision/correlation	Noise	No
Prior strength/producer	Priors	No
Numerical support strategy	Solver	No
Fewer/reordered existing solve-fors	`rows.with_parameters(...)`	No, for a compatible subset
Channels or blackout masks at retained epochs	Observation plan + checked transport	No
Mutual gravity order, initial state or integration step	Model/integrator	Yes
New parameter directions or required output epochs	Parameter map/schedule	Requires appropriate new transport

See reuse and trade studies for exact examples and the distinction between conditioning and marginalization.

CHAPTER 04

Bodies, moments, units and frames

BASCO's public physical inputs use metres, kilograms, seconds and radians. Physical-boundary classes also accept compatible Astropy quantities. Internal native translation is kilometre-based; let the codecs convert it. A plausible numerical answer can still be physically wrong when frames or moment normalization are mixed, so keep these conventions with every input and export.

Analytic ellipsoids

UniformEllipsoid is the simplest complete moment source:

```
import basco as bc

primary = bc.Body(
    "primary",
    bc.UniformEllipsoid(axes=[400.0, 350.0, 300.0], density=2200.0),
)
print(primary.mass)
print(primary.inertia)
```

The axes are three positive **semi-axes**, in metres; density is kg/m³. The distribution generates mass-weighted raw moments through degree 6. It has analytic homogeneous-ellipsoid moments, unlike a finite-resolution triangulated ellipsoid.

`Body.mass` is kilograms and `Body.inertia` contains principal inertias in kg m². The body's display name does not replace the physical-coordinate IDs `primary` and `secondary` assigned by its role in the binary.

Explicit moment distributions

MomentDistribution accepts your own centered principal-axis table. A key `(l, m, n)` represents:

$$T_{lmn} = \int x^l y^m z^n \, dm, \qquad [T_{lmn}] = \mathrm{kg\,m}^{l+m+n}.$$

`(0, 0, 0)` is the mass. First moments and degree-2 products of inertia vanish in the required frame. The three positive diagonal second moments determine principal inertia through sums, for example `Izz = T200 + T020`.

For a safe miniature example, start with a complete analytic table and make the roles explicit:

```
ellipsoid = bc.UniformEllipsoid([400.0, 350.0, 300.0], 2200.0)
distribution = bc.MomentDistribution(
    raw_moments=ellipsoid.moments(order=6),
    reference_radius=ellipsoid.reference_radius,
    bounding_radius=ellipsoid.bounding_radius,
    characteristic_length=ellipsoid.characteristic_length,
    available_degree=6,
    missing="error",
)
primary = bc.Body("primary", distribution)
```

Replace the moment table only when its units, frame, centering and degree completeness are known. `missing="zero"` is an explicit scientific declaration that omitted nonstructural moments are zero. It is not an automatic repair for an incomplete data file.

FIELD	ROLE
`reference_radius`	Fixed radius used for harmonic normalization/reporting
`bounding_radius`	Enclosing sphere used in non-overlap/separation checks
`characteristic_length`	Positive numerical coordinate scale
`available_degree`	Completeness of the supplied nominal moment table

These radii/lengths can differ. Changing a report radius is a convention change that must travel with the coefficients and covariance.

For OBJ ingestion, mesh alignment and fresh shape campaigns, use shapes and Kaula priors. In particular, geometric volume moments are not already mass-weighted moments.

Binary state convention

BinaryState contains six physical fields:

FIELD	MEANING	UNIT
`position`	Inertial secondary-minus-primary position, expressed in primary-body components	m
`velocity`	Inertial relative velocity, expressed in primary-body components	m/s
`omega_primary`	Primary inertial angular velocity, expressed in primary components	rad/s
`omega_secondary`	Secondary inertial angular velocity, expressed in primary components	rad/s
`A_to_N`	Rotation mapping primary-body components to inertial components	dimensionless

FIELD	MEANING	UNIT
`B_to_A`	Rotation mapping secondary-body components to primary-body components	dimensionless

`velocity` is **not** the time derivative of `position` in the rotating primary frame. Similarly, the secondary spin is not supplied in secondary-body components. Rotate those inputs explicitly before building the state.

Both input matrices must be proper rotations. The initial state is a physical input, not an arbitrary 30-element vector. An order-6 mutual model also needs adequate separation relative to the declared body bounds.

System and fixed G

The quickstart's complete `primary`, `secondary` and `state` objects compose as:

```
system = bc.BinarySystem(
    primary, secondary, state,
    G=bc.G_SI,
    name="my binary",
)
model = bc.BinaryDynamics(system, gravity=bc.HouGravity(mutual_order=6))
```

`G` is fixed in this model. Mass reports transform its mass covariance into GM covariance using that fixed value; they do not estimate uncertainty in the gravitational constant.

Native30 and physical18

The retained integration representation has 30 components: translation/spins and two nine-entry rotation matrices. `BinaryState.to_native()` and `BinaryState.from_native(...)` own its codec. Export metadata names each native axis and its unit convention.

Physical state uncertainty has 18 coordinates: relative position, relative velocity, two spins and two three-component local attitude errors. physical_lift and physical_dual map between those tangents and native entries. retract_native applies a finite physical state update using a consistent attitude retraction.

`BinaryState.from_native(native, project=True)` explicitly projects matrix roundoff to SO(3); it is used when passing propagated arc-end states into the next nominal arc. It is not a general license to repair arbitrary invalid attitude inputs or perturb nine DCM entries independently for validation.

Source owners: models.py, coordinates.py, state.py. Continue with estimated parameter coordinates.

CHAPTER 05

Gravity order and estimated parameters

Two independent choices define an experiment: **which gravity terms affect the nominal motion**, and **which physical quantities are uncertain solve-fors**. A degree-6 nominal model does not imply degree-6 estimation. Fixed nominal terms still affect motion and can affect a mass derivative.

Set nominal mutual gravity order

```
model = bc.BinaryDynamics(system, gravity=bc.HouGravity(mutual_order=6))
```

This fragment assumes `system` from the quickstart. HouGravity supports total mutual orders 2, 4 and 6. The distributions must provide complete nominal moment data through the selected order. Keep the gravity order in run labels, configuration and comparison captions.

Mutual order is the truncation of the two-body expansion. It should not be confused with the maximum degree present in a coefficient report, shape ensemble or independent solve-for basis.

Build named independent coordinates

Parameters belongs to a specific model. Its chained calls retain selection order:

```
parameters = (
    bc.Parameters(model)
    .estimate_masses(bodies="both")
    .estimate_moments(bodies="both", degrees=(2,))
    .estimate_harmonics(bodies="both", degrees=(3, 4))
    .estimate_initial_state()
)
resolved = parameters.resolve()
print(resolved.labels)
print(resolved.units)
print(resolved.scales)
```

`bodies` can be `"primary"`, `"secondary"` or `"both"`. Omitted quantities are fixed at the supplied nominal. Omit `estimate_initial_state()` only when a conditional, fixed-initial-state analysis is intended.

SELECTION	INDEPENDENT PHYSICAL COORDINATES	UNIT
`estimate_masses()`	Each selected mass	kg
`estimate_moments(degrees=(2,))`	Three diagonal mass-normalized moments per selected body	m^2
`estimate_moments(degrees=(3, 4))`	Complete normalized raw degree-3/4 moments, including trace directions	m^3 / m^4
`estimate_harmonics(degrees=(3, 4))`	Independent unnormalized exterior C/S coefficients with a declared trace complement	dimensionless
`estimate_initial_state()`	18 physical initial-state errors	Mixed SI/radians

Raw and harmonic selections cannot overlap for the same body and degree. Degree-2 harmonic-only selection is rejected because the rotational equations need the third inertia coordinate. Independent degree-5/6 solve-fors, and solve-for degrees above the active gravity order, are unsupported.

Raw versus exterior parameterizations

The paper reproduction uses:

```
parameters = (
    bc.Parameters(model)
    .estimate_masses()
    .estimate_moments(degrees=(2, 3, 4))
    .estimate_initial_state()
)
```

There are 58 physical solve-fors: two masses plus 28 selected normalized moments per body. Coestimating the initial state adds 18, and a three-component barycenter offset/rate adds 6 observation-nuisance coordinates: **82 joint coordinates**.

The mixed degree-2/raw plus degree-3/4/harmonic basis in the first example has 40 physical coordinates: two masses, three degree-2 moments per body, and 7 degree-3 plus 9 degree-4 coefficients per body. With the same state/barycenter treatment it has **64 joint coordinates**.

These are different inverse problems. Raw higher-degree moments can contain exterior trace null directions. Priors that constrain such directions must be stated. The exterior coordinate map uses a fixed trace-complement gauge, rather than making the missing raw directions implicitly measurable.

Parameter normalization and mass pathways

Raw solve-for coordinates are normalized by body mass, not radius. A degree- `n` normalized raw moment has units m^n. Harmonics are dimensionless at a fixed reference radius. Their public report uses unnormalized C/S conventions; Kaula ensembles use a separately declared normalized convention and map it explicitly.

Changing a mass scales all retained mass-weighted moments of that body at fixed normalized shape, including higher nominal moments that are not independently estimated. `ResolvedParameters.raw_jacobian` records this full physical map; `raw_slots` records raw moment order. Do not implement a custom mass variation by changing only the monopole.

Scaling and layouts

Positive characteristic scales support numerical conditioning even when a coefficient's nominal value is zero. They introduce no statistical knowledge. CoordinateLayout records ordered labels, units, scales and `n_state`, the prefix count of local state/nuisance coordinates.

Use `resolved.labels`, the joint `rows.layout`, and named covariance access rather than depending on numeric positions. Results explains `covariance_for(...)` and null-aware derived transforms.

Change degree after propagation

When a retained linearization already contains the needed directions, a compatible subset can be selected with `rows.with_parameters(...)`. The omitted coordinates are then **conditioned on their nominal values**; they are not marginalized. Priors reset unless supplied explicitly for the new layout.

Changing nominal gravity order always changes dynamics and requires new matching transport. Reuse and trades gives examples of both operations.

The Hera degree-trade example distinguishes the paper's primary-only higher-degree no-prior case from a symmetric both-body experiment. Preserve that distinction when comparing Figure 5.

CHAPTER 06

Observations, barycenter and schedules

An ObservationPlan describes predicted measurement directions, their default uncertainties and their times. It is independent of the gravity kernel, derivative provider and covariance solver. Ordinary plans currently use positive elapsed epochs on one uniform grid.

The snippets below use `import basco as bc` and NumPy as `np`. Their `model` and `parameters` come from the quickstart.

Built-in measurement channels

CHANNEL	PREDICTION DIRECTIONS	DEFAULT/PUBLIC UNITS
RelativePosition	Three inertial components of secondary-minus-primary position	m
BodyAttitude	Three local inertial attitude prediction directions for primary or secondary	rad
PrimaryBarycentricPosition	Three inertial primary-position components relative to the binary barycenter, plus optional nuisance	m

```
channels = (
    bc.RelativePosition(sigma=1.0),
    bc.BodyAttitude("primary", sigma=np.deg2rad(1.0)),
    bc.BodyAttitude("secondary", sigma=np.deg2rad(1.0)),
    bc.PrimaryBarycentricPosition(
        sigma=1.0,
        nuisance=bc.PolynomialBias("barycenter", degree=1),
    ),
)
plan = bc.ObservationPlan.uniform(
    stop=86400.0,
    cadence=900.0,
    channels=channels,
)
```

Each built-in channel takes a scalar sigma. For component-specific weights, supply `DiagonalNoise` in the assembled row order, or implement a custom channel. Each sigma must be positive and finite. Component labels across channels must be unique. A separate noise model can replace these default independent weights after

linearization.

`BodyAttitude` supplies the covariance tangent. A general adapter for actual measured-attitude residuals is unfinished. Adding a covariance channel is not equivalent to completing observed-data fitting for that channel.

Didymos position relative to the center of mass

For inertial relative position `rho_N`, the primary's prediction is:

$$\boldsymbol{y}_A(t) = -\frac{M_B}{M_A + M_B}\boldsymbol{\rho}_N(t) + \boldsymbol{R}_0 + (t - t_{\mathrm{ref}})\boldsymbol{V}_0.$$

At fixed relative state, its direct mass derivatives include:

$$\frac{\partial \boldsymbol{y}_A}{\partial M_A} = \frac{M_B}{(M_A + M_B)^2}\boldsymbol{\rho}_N, \qquad \frac{\partial \boldsymbol{y}_A}{\partial M_B} = -\frac{M_A}{(M_A + M_B)^2}\boldsymbol{\rho}_N.$$

The implementation retains these **direct** terms in addition to propagated mass sensitivity. observations.py owns the value and derivative maps. This is the modular channel used in the reproduced Hera study.

Omitting `nuisance` conditions on a known barycenter. PolynomialBias with `degree=1` instead coestimates a free three-component offset and rate, adding six unknowns without an implicit prior. Its `reference_epoch` is seconds. Polynomial terms use `(t - t_ref)**k / k!`, so degree-2 coefficients represent acceleration in m/s² rather than an arbitrary quadratic coefficient.

Epochs and row order

The first measurement is at one cadence after arc start; there is no observation at `t=0`, and no invented final observation when duration is not on the cadence boundary. Integration still retains its own initial state and the output states needed by the observation plan.

```
print(plan.epochs)
print(plan.visibility())
print(plan.row_sigmas())
```

Observation rows are ordered by epoch, then visible channels in the supplied order, then components within each channel. `rows.row_labels` and `rows.row_units` are authoritative after assembly. Correlated covariance and residual vectors must follow that exact order.

Add a recurring blackout

```
plan = bc.ObservationPlan.uniform(
    stop=14 * 86400.0,
    cadence=900.0,
    channels=channels,
    schedule=bc.PeriodicBlackout(period=86400.0, duration=8 * 3600.0),
)
```

This removes observations during the blackout, while the binary continues propagating. A schedule masks the existing output grid; it does not create new epochs or accelerate dynamics by skipping gaps.

Give one stream a different cadence

```
channels = (
    bc.RelativePosition(sigma=1.0),
    bc.Scheduled(
        bc.BodyAttitude("secondary", sigma=0.01),
        bc.Every(period=1800.0),
    ),
)
```

With a 900-second plan grid, `Every(1800.0)` retains every other matching epoch for that stream. The plan schedule and stream schedule must both permit an observation.

SCHEDULE	MEANING
Always	All existing epochs
Every	Epochs matching a repeated period/offset
Periodic	Observed intervals of a repeated period
PeriodicBlackout	Complement of repeated blackout intervals
Windows	Union of half-open `[start, stop)` observing windows
AllOf, AnyOf, Not	Combine schedules, also via `&`, `|` and `~`

```
passes = bc.Windows(((3600.0, 7200.0), (14400.0, 18000.0)))
allowed = passes & bc.PeriodicBlackout(period=86400.0, duration=1800.0)
```

`Windows.around(centers, before=..., after=...)` builds pass windows around supplied centers. Schedules evaluate elapsed mission time, including `schedule_offset`; multi-arc composition supplies each arc's offset so a daily gap does not accidentally restart every arc.

Reuse dynamics or add a new measurement

Changing channels or masks on the same retained grid can reuse checked transport. See reuse and trades. A new measurement can implement the ObservationChannel structural contract without inheriting a base class. Its local prediction derivative and direct parameter derivative must both be explicit; a missing derivative is not assumed zero.

The extension chapter includes a range-channel example and the boundaries of custom-observation AD. Multi-arc studies explain continuous nominal trajectories with independent local estimated states.

CHAPTER 07

Noise, priors and numerical support

The observing geometry supplies information directions; noise determines their weight; priors add independent stated knowledge; the solver determines finite-precision support. Keep these four roles separate when comparing experiments.

Snippets assume the `study`, `rows`, `primary`, NumPy and `bc` objects from the quickstart.

Independent measurement noise

Without an override, channel sigmas define independent scalar observation errors. After propagating once, replace that model with DiagonalNoise:

```
baseline = rows.covariance()
twice_the_sigma = rows.covariance(
    noise=bc.DiagonalNoise(2.0 * rows.sigmas),
)
```

This changes standard deviations, not variances, and performs no new dynamics. `rows.sigmas` follows the actual retained observation rows, including schedule masks.

Correlated measurement noise

CorrelatedNoise accepts a finite symmetric positive-definite covariance and uses Cholesky whitening. Its rows and columns follow the full scalar observation order and mixed physical row units.

For a small study, this example adds correlation only between its first two scalar rows:

```
measurement_covariance = np.diag(rows.sigmas**2)
measurement_covariance[0, 1] = measurement_covariance[1, 0] = (
    0.2 * rows.sigmas[0] * rows.sigmas[1]
)
correlated = rows.covariance(
    noise=bc.CorrelatedNoise(measurement_covariance),
)
```

This is an API illustration, not a recommended physical correlation model. A dense full observation covariance can be large. Custom NoiseModel implementations may provide appropriate structured whitening, but must whiten vectors and matrices consistently and fully describe their settings.

A named Gaussian prior

GaussianPrior identifies independent coordinates by labels and units:

```
mass_prior = bc.GaussianPrior(
    name="independent primary mass knowledge",
    labels=("primary.mass",),
    units=("kg",),
    covariance=[[(0.05 * primary.mass)**2]],
    source="external mass constraint",
)
posterior = rows.covariance(priors=(mass_prior,))
```

The example states a 5% prior standard deviation. It does not alter what the observations measured. Replacing `priors` replaces the complete retained set for that analysis; it does not silently append to it. Use `priors=()` for data-only analysis.

A multi-coordinate prior preserves off-diagonal covariance. A zero covariance is not a way to fix an unknown; remove the solve-for when conditioning on an exact value. Unknown labels, wrong units and duplicate prior information identities are errors.

Mean and reference matter for estimation

For covariance alone, an omitted prior mean centers the prior on the nominal. For repeated nonlinear estimation, use a fixed explicit physical prior mean and update the reference at each iterate. Centering a prior on every new iterate changes the statistical problem.

For example, a fixed prior mean can be written:

```
mass_prior = bc.GaussianPrior(
    "external primary mass",
    labels=("primary.mass",),
    units=("kg",),
    covariance=[[(0.05 * primary.mass)**2]],
    mean=[primary.mass],
)
```

PriorBlock supports knowledge about mapped physical quantities. It carries a Jacobian, column and row layouts, a noise model, mean/reference and source. IndependentPriors composes independent producers while retaining each block's correlations and provenance.

KaulaPrior is a reusable physical prior producer, explained in shapes and Kaula. Multi-arc shared priors belong once on the global study, not once per arc.

Joint square-root solver

SquareRootSolver is the default. It uses scaled SVD support and QR covariance/correction algebra. It diagnoses data and posterior support, retains null directions and returns full joint covariance only when supported.

```
joint = rows.covariance(solver=bc.SquareRootSolver())
print(joint.summary())
```

Characteristic scales are physical column scales for conditioning. They are not prior sigmas. Zero nominal coefficients still need positive scales. Inspect `rows.layout.scales` and the singular spectrum when rank is sensitive.

Nuisance-projected solver

NuisanceProjectedSolver eliminates the local state/nuisance block with Householder QR before diagnosing marginal physical information. It matches the documented Hera convention:

```
projected = rows.covariance(solver=bc.NuisanceProjectedSolver())
```

This strategy requires numerically usable local columns and applies an explicit nuisance-condition guard. It does not make an unobservable local block safe to eliminate. Joint and projected strategies can make different declared support decisions in ill-conditioned problems; a different finite answer is a reason to inspect diagnostics, basis and refinement rather than choose the prettier result.

Both strategies preserve prior provenance and expose common support diagnostics, including `numerical_support_loss_after_prior`. This diagnostic can identify loss/rotation of numerically retained data support after adding prior rows; a mathematically informative prior does not guarantee every finite-precision truncation behaves ideally.

Rank thresholds are scientific reporting inputs

`rank_rtol` controls a numerical support threshold. It is not a prior and does not repair observability. Record the selected policy with results, and compare thresholds/bases/refinement when conclusions depend on it. Never turn a threshold until an expected sigma appears.

The next chapter, reading results, explains data versus posterior rank, supported quantities and full cross covariance. The solver source and projected source expose the actual support policies.

CHAPTER 08

Reproduce the Hera paper

The Hera study is BASCO's main scientific example. It estimates **both masses and gravity-related moments through degree 4**, propagating the nominal binary with **Hou mutual gravity order 6**. Four rotational states reproduce every one of the paper's **36 printed Table 2 percentages**. This chapter gives the runnable entry points, the scientific choices behind them, and what the reproduction establishes.

Start with installation and your first run if the managed environments are not built. The examples gallery offers smaller demonstrations before committing to full propagation.

Run one case or all four

Run these commands from the repository root after the managed build:

```
# Full tumbling, tables and publication-style figures.
python3 -B tools/dev.py hera

# All four paper cases; independent processes; tables and numeric exports.
python3 -B tools/dev.py hera_cases

# Select cases and control concurrency.
python3 -B tools/dev.py hera_cases AC1 AC3 --jobs 2

# Add figures to the four-case runner in the plotting environment.
python3 -B tools/dev.py run --plot examples/hera_all_cases.py --figures
```

The one-case runner is examples/hera_covariance.py. The four-case runner is examples/hera_all_cases.py. Both compose ordinary library components; neither contains a second covariance engine.

Full cases cost minutes with AD on the measured Mac; historical native runs took about 25 minutes per case. Phase 2's four AD cases each took 199–201 seconds while other validation shared the host. Compilation, CPU, workload and scientific choices affect timing. The four-case runner supports `--jobs 1` to reduce simultaneous resource use. It does not make a short run a paper reproduction: Table 2 requires the full 16 orbits and the stated settings.

`hera` requests external LaTeX through `PlotTheme(tex=True)`. It needs both the plotting dependencies and a working TeX installation. The four-case runner uses the theme's default math rendering unless changed. See results and plotting.

The scientific contract

CHOICE	PAPER REPRODUCTION
Primary	Didymos DART SPC v003 polyhedral moments, complete through degree 6
Secondary	Uniform ellipsoid, semiaxes 96, 74 and 59 m, complete through degree 6
Masses	Didymos 525,852,763,514.02 kg; Dimorphos 4,539,936,844.07 kg
Fixed gravitational constant	`6.674e-11` $m^3\ kg^{-1}\ s^{-2}$
Fixed reference radii	367.8508020568785 m and 74.83733635110467 m
Mutual gravity cutoff	Hou total order 6
Independent physical solve-fors	Both masses; 28 mass-normalized raw moments per body: 3 degree-2 diagonal terms, 10 degree-3 terms, 15 degree-4 terms
Local coordinates	18 initial-state errors; 3 barycenter offsets and 3 barycenter rates
Total unknowns	58 physical + 18 initial state + 6 barycenter = **82**
Duration	16 × 11.3674 h = 654,762.24 s
Integration	Fixed-step RK4, 75 s
Measurement epochs	727 positive epochs at 900 s cadence; no epoch-zero observation
Scalar design rows	12 per epoch, **8,724** in total
Posterior prior	36 harmonic rows plus 56 independent raw-moment rows
Unconstrained by that prior	Both masses, all initial-state errors, all barycenter coordinates
Numerical solver	`NuisanceProjectedSolver`
Current default derivative route	Hou automatic differentiation; native analytic is explicit and preserved

The radii are the fixed bounding-ellipsoid volume radii used in the manuscript. Substituting a mesh-volume radius changes harmonic normalization. The nominal moments, gravity cutoff, uncertain coordinates and output normalization are independent choices; parameters explains why.

`hera.load_inputs` reads checksum-verified BASCO-owned inputs from reference/hera-20261016. The original PyGUBAS and Gravity Inertia directories are not runtime dependencies. The benchmark's preprocessing and state construction live in basco.benchmarks.hera.

What each rotational case means

CASE	NAME	INITIAL SECONDARY LIBRATION	WOBBLE	ORBIT AND EXCITATION
AC0	Equilibrium	0°	0°	Circular, planar, synchronous; retained order-6 speed calibration
AC1	Libration only	10°	0°	Eccentricity 0.0274; primary inclination 1°; secondary obliquity 3°
AC2	Limited tumbling	10°	20°	Same excited orbit as AC1
AC3	Full tumbling	90°	43.09°	Same excited orbit as AC1

The initial-condition helper retains the source full-field period/separation solve and the source spin conventions. The secondary's excited-case spin uses the pre-impact 11.921493 h period. Changing the active gravity order keeps these order-6-calibrated initial conditions unchanged. It does not silently refit an orbit.

A readable modular runner

This complete composition is the paper's default study, using the public components. `ROOT` below is the absolute BASCO checkout path; when writing a script inside `examples/`, use `Path(__file__).resolve().parents[1]` instead of hard-coding it.

```python
from pathlib import Path

import numpy as np
import basco as bc
from basco.benchmarks import hera

ROOT = Path("/absolute/path/to/BASCO")
REFERENCE = ROOT / "reference" / "hera-20261016"

# 1. Nominal bodies, rigid-body state and active Hou gravity order.
inputs = hera.load_inputs(REFERENCE)
model = hera.build_hera_model(inputs, "AC3", gravity_degree=6)

# 2. Uncertain physical coordinates and the initial-state tangent.
parameters = (
    bc.Parameters(model)
    .estimate_masses()
    .estimate_moments(degrees=(2, 3, 4))
    .estimate_initial_state()
)

# 3. Measurement streams and independently estimated barycenter.
observations = bc.ObservationPlan.uniform(
    stop=hera.DURATION_S,
    cadence=900.0,
    channels=(
        bc.RelativePosition(sigma=1.0),
        bc.BodyAttitude("secondary", sigma=np.deg2rad(1.0)),
        bc.BodyAttitude("primary", sigma=np.deg2rad(1.0)),
        bc.PrimaryBarycentricPosition(
            sigma=1.0,
            nuisance=bc.PolynomialBias("barycenter", degree=1),
        ),
    ),
)

# 4. Prior knowledge, integration and numerical support policy.
study = bc.CovarianceStudy(
    model,
    parameters,
    observations,
    integrator=bc.RK4(dt=75.0),
    priors=(hera.HeraKaulaPrior(parameters, REFERENCE),),
    solver=bc.NuisanceProjectedSolver(),
)

# 5. Propagate once, analyze, then present without new dynamics.
rows = study.linearize()
result = rows.covariance()
paper = hera.hera_report(result)
print(paper.summary())

folder = bc.export_study(
    rows,
    result,
    ROOT / "results",
    label="hera-full-tumbling-g06-e04",
    reports={
        "paper": paper,
        "harmonics": result.harmonics(),
        "mass": result.mass_properties(),
    },
    figures=True,
    theme=bc.PlotTheme(tex=False),
```

```
)
print(folder)
```

Each numbered part has one owner. BinaryDynamics describes the physics; Parameters selects independent uncertainty coordinates; ObservationPlan describes the measurements; HeraKaulaPrior produces ordinary prior rows; NuisanceProjectedSolver owns numerical elimination; export_study owns standardized output. The derivative provider is another component: add `transport=bc.NativeAnalyticTransport()` to the study for native analytic partials, keeping everything else identical.

The factory `make_study` in the actual runner exposes `state`, `gravity_degree`, `estimate_degrees`, `parameterization`, `higher_degree_bodies`, `prior`, `dt`, `duration` and `transport`. It is an example function, not a hidden global setting or a second public package API.

The Didymos position measurement is essential

The final paper includes Didymos' position relative to the system center of mass. Its model is

$$\boldsymbol{y}_A(t) = -\frac{M_B}{M_A + M_B}\, C_{NA}(t)\boldsymbol{r}_A(t) + \boldsymbol{R}_0 + (t - t_{\mathrm{ref}})\boldsymbol{V}_0.$$

PrimaryBarycentricPosition includes the direct mass-ratio derivative **and** the mass dependence carried through the trajectory. The PolynomialBias adds the six free position/rate coordinates. Dropping this channel changes the experiment. Keeping the channel but dropping its nuisance model assumes a known barycenter and also changes the experiment.

The four streams contribute three scalar components each: inertial relative position, secondary attitude tangent, primary attitude tangent, and primary barycentric position. Position noise is 1 m per component; attitude noise is 1° per component, expressed internally in radians. These attitude directions support formal covariance; general measured-attitude residual ingestion remains separate work.

What the Kaula prior actually adds

The paper configuration combines two **independent** information components: 36 harmonic rows and 56 raw-moment rows. It does not add a mass prior. HeraKaulaPrior binds the reusable KaulaPrior and archived, checksum-verified laws to the selected layout.

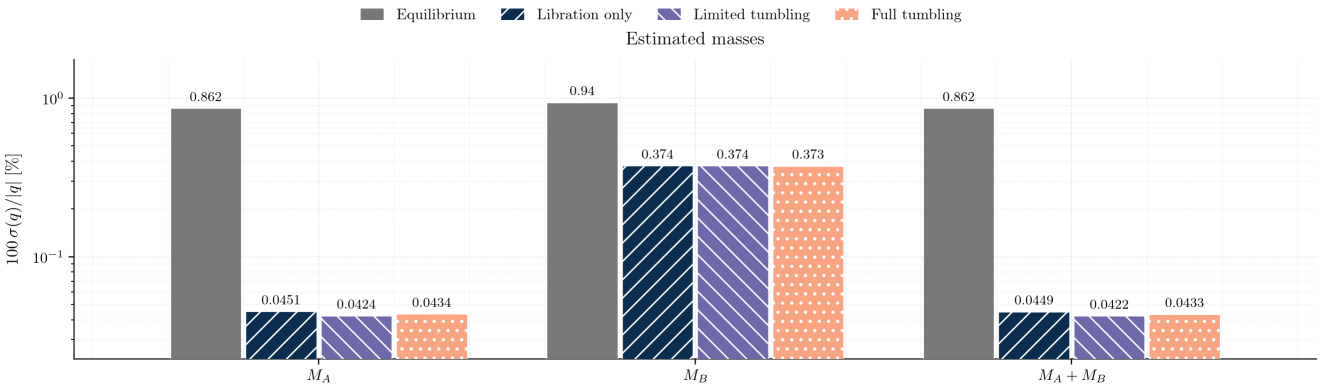
The paper's plotted harmonic prior uncertainty is the original Kaula envelope. Its normalized polar-moment prior uses the combined degree-2 prior covariance. These are distinct reported quantities. Figure 3 comes from refitting the archived coefficient clouds; it does not imply that every original random mesh was regenerated. See shapes and Kaula for newly sampled ensembles.

Table 2: all printed percentages

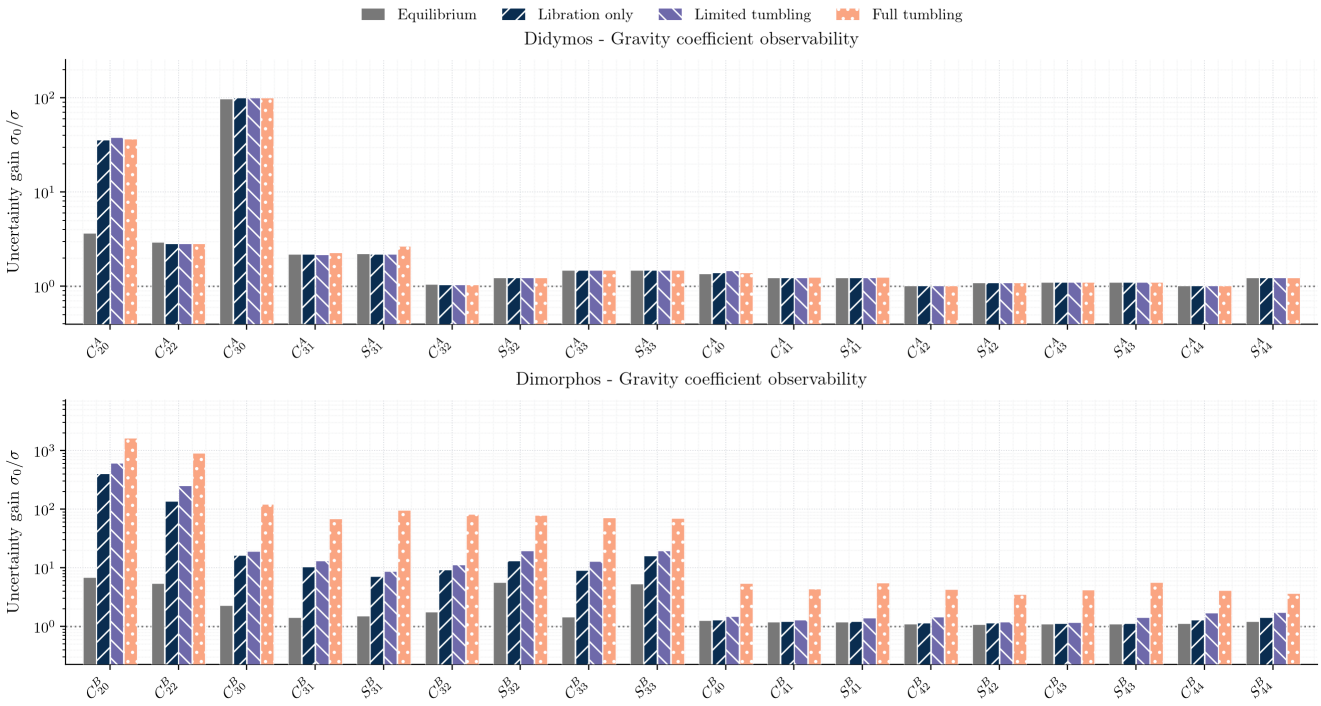
Every entry is formal `100 × sigma / abs(nominal)` in percent. The paper's `Izz` means the dimensionless `Izz/(M R²)`. hera_report also handles the paper's 4π geodesy-normalized harmonics; the general `result.degree2()` report uses unnormalized degree-2 coefficients and dimensional inertia. Compare the same conventions.

FORMAL 1Σ [%]	EQUILIBRIUM	LIBRATION ONLY	LIMITED TUMBLING	FULL TUMBLING
Didymos C20	51.9	5.29	5.00	5.20
Didymos C22	30.5	31.4	31.5	31.5
Didymos C30	13.7	13.4	13.3	13.4
Didymos Izz/(M R²)	18.0	16.9	16.9	16.9
Didymos mass	0.862	0.0451	0.0424	0.0434
Dimorphos C20	29.3	0.502	0.336	0.123
Dimorphos C22	23.8	0.955	0.513	0.143
Dimorphos Izz/(M R²)	21.5	0.449	0.288	0.119
Dimorphos mass	0.940	0.374	0.374	0.373

Download the published Table 2 CSV. The four-case runner prints the cases side by side and marks any value outside the manuscript's printed rounding interval.



This retained gallery uses the paper's blue–purple–orange family. These are existing validated assets, not newly propagated results generated by viewing this documentation.



Degree, prior and gravity trades

Use separate choices rather than changing a kernel:

```
# Inside a script with make_study from examples/hera_covariance.py:
degree2 = make_study("AC3", estimate_degrees=(2,), prior="none")

# The paper's expanded no-prior case: primary-only higher harmonics.
primary_high = make_study(
    "AC3",
    estimate_degrees=(2, 3, 4),
    parameterization="harmonics",
    higher_degree_bodies="primary",
    prior="none",
)

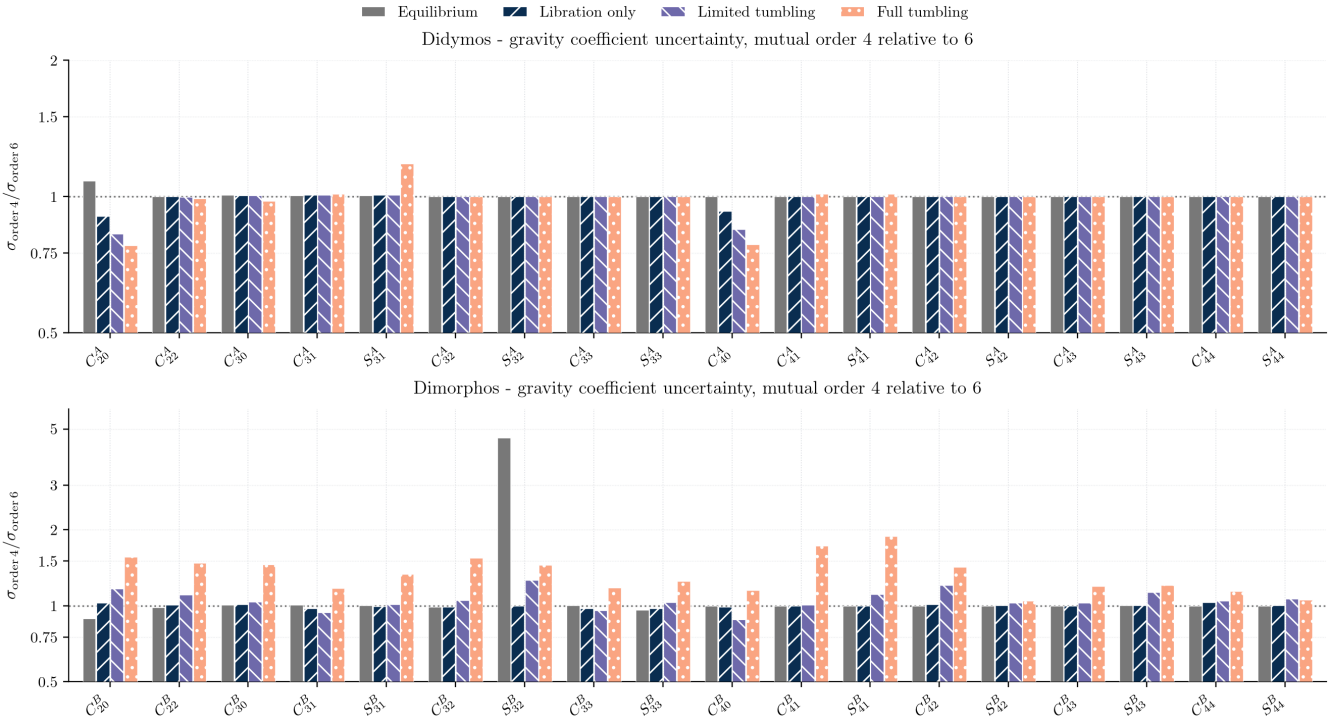
paper_baseline = make_study("AC3")
gravity4 = make_study("AC3", gravity_degree=4)
```

The second Figure 5 group estimates both bodies' degree-2 inertia coordinates and masses, plus **primary-only exterior degree-3/4 harmonics**. It is not the same as both-body raw degree-4 moments without a prior. The latter retains structural trace null directions; its unavailable uncertainties must remain unavailable.

hera_degree_trades.py implements these comparisons. `propagate` makes a new transport; `compare` verifies and remaps completed BASCO transports without more propagation. Retained comparison fixtures are native, so `compare` defaults to `--derivatives native`; fresh `propagate` defaults to AD. A backend identity mismatch is rejected rather than relabeled.

Figure 5's reproduced no-prior sigma errors are 2.5417e-6 for degree 2 and 3.8168e-6 for the primary-only higher-degree experiment. The four prior cases pass the same Table 2 checks. Degree-trade evidence records the exact scientific configuration.

Changing gravity order requires a new trajectory. Changing only compatible priors/noise can reuse rows. Reducing the uncertain coordinate subset can reuse validated transport through the corresponding map; it conditions omitted solve-fors as fixed. The workflow chapter explains reuse boundaries.



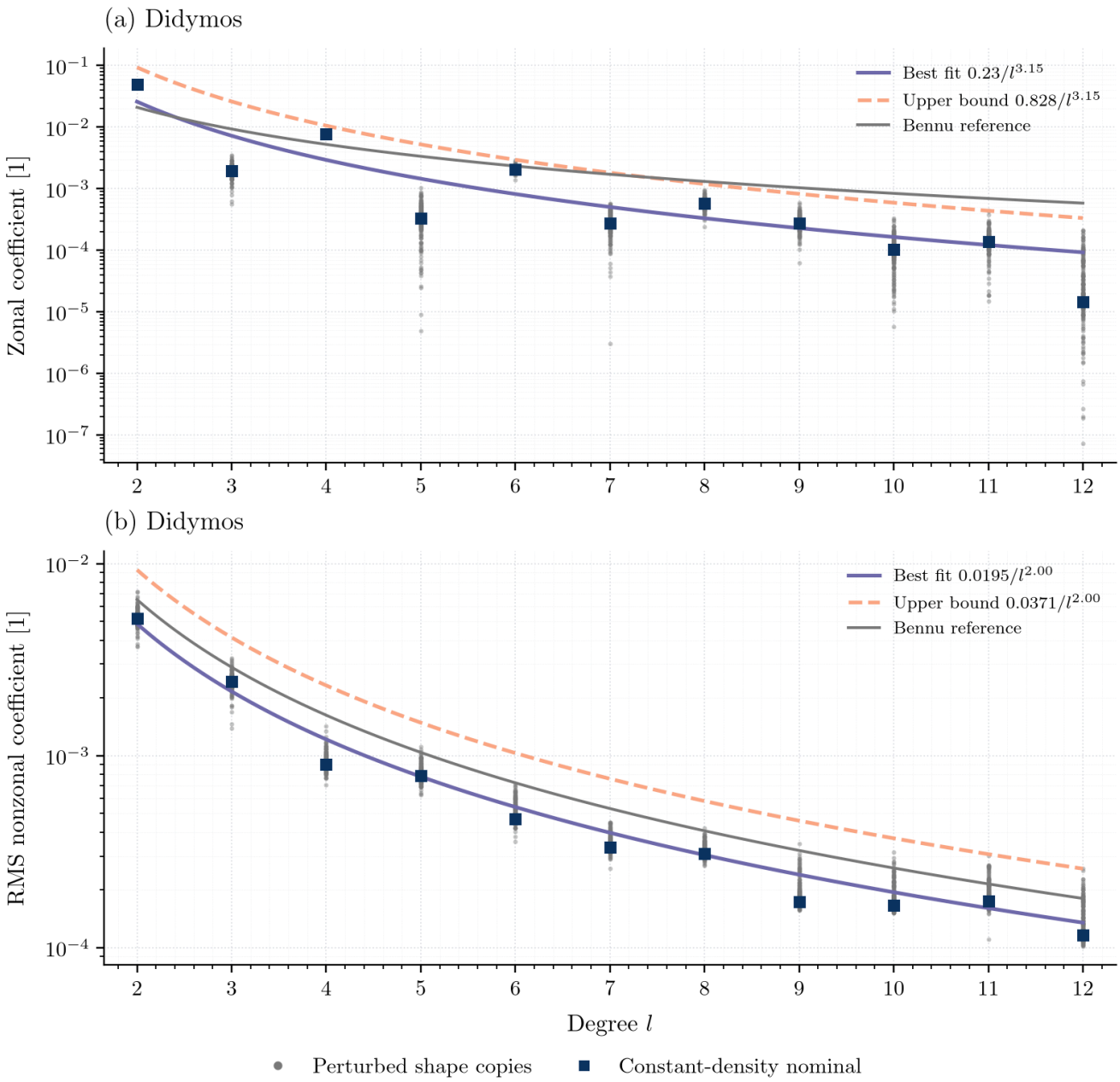
An order-4/order-6 ratio compares two **model-conditioned covariances**. It does not measure parameter bias from omitted gravity or prove that the model with smaller formal errors is more accurate.

Figure 3 and the shape ensembles

```
# Refit both archived clouds; no trajectory propagation or new mesh ensemble.
python3 -B tools/dev.py run --plot examples/hera_kaula.py --tex

# Explicitly sample a small new seeded demonstration instead.
python3 -B tools/dev.py run --plot examples/hera_kaula.py \
  --fresh --max-degree 4 --copies 3 --subdivisions 1
```

The archived fit reproduces the Didymos law exactly and the Dimorphos law to 5.9e-15 maximum relative difference. The ensemble includes 200 shape perturbations plus density variants. Fresh low-resolution sampling is an API demonstration, not a fresh reproduction of the complete paper ensemble.



Fresh propagation, row replay and saved results

OPERATION	WHAT IT ESTABLISHES	ADDITIONAL DYNAMICS
Fresh `CovarianceStudy.linearize()`	BASCO's transport, observation assembly, priors and solver reproduce the configured experiment	Yes
`hera.load_hera_archive(...)`	Archived PyGUBAS rows, transformed into physical SI coordinates, agree under BASCO covariance/report algebra	No

OPERATION	WHAT IT ESTABLISHES	ADDITIONAL DYNAMICS
`hera.load_retained_transport(...)`	A checked completed BASCO trajectory supports a compatible new solve-for/measurement analysis	No
`read_run(folder)` or presentation rendering	A saved result is checksum-valid and can be read/reformatted	No

Replay converts native kilometers, fractional moment/GM columns and centered Legendre barycenter coefficients into SI mass, moment, `R0` and `V0` coordinates. It is valuable independent evidence of conversions and algebra, but it cannot validate fresh dynamics on its own.

The latest Phase 2 acceptance is **four fresh default-Hou-AD runs**, not replay. All 160 reported sigmas pass the declared 1e-3 relative archive/native criterion, and all 36 printed Table 2 values match their rounding bins:

CASE	MAXIMUM RELATIVE SIGMA ERROR VS ARCHIVE	VS FROZEN NATIVE
AC0	5.291134e-11	1.160849e-11
AC1	6.788903e-11	3.926393e-11
AC2	2.780115e-10	1.373615e-10
AC3	2.243221e-6	1.457388e-6

The full-tumbling case amplifies long-arc rounding; its small residual difference is recorded, not hidden. Earlier native fresh propagation and archived-row replay are separate checkpoints. Historical replay's maximum relative sigma error was 2.475084e-9 under its stricter 1e-7 criterion.

The latest Phase 2 validation record owns the current acceptance numbers. Frozen compact evidence retains covariance/scales, comparisons, executed source and logs. Earlier reproduction notes explain source capture, conversions and native evidence; statements there about then-unimplemented AD/schedules are historical.

Read the outputs without losing the experiment

The one-case export contains the `paper`, `harmonics` and `mass` report families, numeric arrays, layouts/units, configuration/provenance, checksums and optional figures. Standard run names include a descriptive label, UTC timestamp and analysis identity. See results for the full directory contract.

```
saved = bc.read_run(folder)  # Verifies the completed standard run.
print(result.mass_properties().summary())  # Both masses, total, GMs and ratio.
print(hera.hera_report(result).summary())  # Exact paper conventions.
```

Never label a prior/noise/degree variant as the paper baseline without recording its changes. The saved configuration and provider identity are the evidence, not the folder's informal nickname.

Limits of the reproduction

These are formal local uncertainties around a nominal rigid-body model. They do not constitute a fit to actual Hera observations, a nonlinear uncertainty-validity proof over chaotic arcs, or a new integration-step convergence study. Complete end-to-end finite-difference coverage of the covariance chain is still open.

Round 20 supplies Table 2's baseline. Six same-dynamics round-21 prior/noise/baseline variants have been checked; **13 other round-21 trajectory/basis variants remain unverified**. In particular, one archived position-noise sweep holds the barycentric noise fixed while later paper plotting ties it to relative-position noise. Baseline 1 m results coincide; other points need the matching reanalysis.

Continue with multi-arc studies, shapes and Kaula, or the complete capability map.

CHAPTER 09

Results, plots and exported files

Read uncertainty availability before reading its numerical value. BASCO separates quantities supported by the experiment, quantities fixed by configuration and unresolved directions. Tables, plots and exports all consume the same canonical presentation so those distinctions remain consistent.

Snippets use the `rows` and `result` produced by the quickstart.

Start with the summary

```
print(result.summary())
print(result.mass_properties().summary())
print(result.degree2().summary())
print(result.harmonics(max_degree=4).summary())
```

CovarianceResult retains data rank, posterior rank, marginal physical rank, singular values, thresholds, nullspace and diagnostics. Priors may make a posterior supported without making its data-only counterpart observable.

REPORT	CONTENTS AND CONVENTION
`mass_properties()`	Individual and total mass, fixed-G individual/total GM, secondary/primary mass ratio
`degree2()`	Normalized moments, unnormalized C20/C22 and dimensional Izz
`harmonics(max_degree=4)`	Unnormalized exterior C/S at the model's declared fixed reference radii

DerivedReport holds labels, values, units, Jacobian, covariance, row statuses, conventions and `sigma`. A fixed quantity with zero error is distinct from a measured quantity with high precision. A zero nominal coefficient can have a meaningful absolute sigma but has no defined relative percentage uncertainty.

Keep cross covariance

Mass sum, mass ratio and gravity transforms need the relevant off-diagonal terms. Do not combine individual sigma values as if independent unless independence is part of the actual covariance.

```
if result.full_rank:
    labels = ("primary.mass", "secondary.mass")
    mass_covariance = result.covariance_for(labels)
    mass_correlation = result.correlation_for(labels)
```

These named methods preserve the requested order and return protected copies. `parameter_covariance` is the physical-parameter block; `state_parameter_covariance` retains the local-state/nuisance to physical-parameter cross block when available.

For a deficient joint solution, `joint_covariance` is unavailable. `supported_subspace_covariance` is only valid for functions that annihilate the reported nullspace. It is not a finite full covariance for arbitrary unresolved coordinates.

Define a custom derived quantity

This fragment computes total mass from the full named coordinate layout, using the same null-aware transform as built-in reports:

```
labels = rows.layout.labels
jacobian = np.zeros((1, len(labels)))
jacobian[0, labels.index("primary.mass")] = 1.0
jacobian[0, labels.index("secondary.mass")] = 1.0
total = result.derived(
    labels=("custom.total_mass",),
    values=[study.model.system.primary.mass + study.model.system.secondary.mass],
    units=("kg",),
    jacobian=jacobian,
    conventions={"definition": "primary.mass + secondary.mass"},
)
print(total.summary())
```

It assumes both masses were selected and NumPy is imported. Normally use `mass_properties()` for this quantity; the example shows the public custom-transform boundary. The Jacobian columns must match the entire joint layout, including local columns.

Export a complete run

```
folder = bc.export_study(
    rows,
    result,
    root=Path(__file__).resolve().parent / "results",
    label="my-binary-g06-e04",
    reports={
        "mass": result.mass_properties(),
        "harmonics": result.harmonics(4),
    },
    figures=True,
    theme=bc.PlotTheme(tex=False),
)
```

This needs `Path` from `pathlib` and the plotting environment. Omit `figures=True` for a numeric/table-only run. The standard folder name includes a readable label, UTC timestamp and scientific analysis digest.

```
my-binary-g06-e04__UTC-TIMESTAMP__DIGEST/
  README.md
  configuration.json
  run.json
  manifest.json
  reports/
    report.md
    report.json
    tables/
    figures/
    manifest.json
  arrays/
    metadata.json
    *.npy
  validation/
    README.md
```

`configuration.json` stores configuration. `run.json` names matrix axes, layouts, units, identities, diagnostics and prior provenance. Plan sigmas are retained separately from the actual analysis weighting. Numeric arrays include states, sensitivities, rows, scales, covariance when available and the rank spectrum/nullspace.

Files are staged, checksummed and published together. Existing destinations are refused. Export does not automatically launch derivative checks: the validation directory states what is known, and checks remain explicitly requested.

The analysis digest retains scientific/statistical choices and scales but excludes elapsed runtime. Creation time remains in folder names/metadata; timing remains available for operational evidence.

Reload without dynamics

```
arrays, presentation, run = bc.read_run(folder)
H = arrays["design_rows_SI"]
print(run["metadata"]["layouts"]["joint_coordinate"]["labels"])
bc.write_presentation(
    presentation,
    "restyled-results",
    figures=True,
    theme=bc.PlotTheme(tex=False, width=10.0, font_size=11.0, dpi=300),
)
```

read_run verifies checksums and returns data, not a reconstructed live study. read_presentation reloads just a canonical report. A damaged manifest, incomplete run or escaping manifest path is an error.

For standalone numeric/string evidence, write_artifact and read_artifact provide verified JSON/NPY folders. Object arrays and pickle are refused. Each array needs meaningful names and accompanying axis/unit metadata supplied by the producer.

One plotting theme

PlotTheme owns widths, DPI, font sizes, math text/external LaTeX, grids, palette and PNG/SVG/PDF formats. Standard figures use the paper-inspired navy/blue, purple and orange scheme. Rendering uses a local Matplotlib context, not global import-time style changes.

`tex=False` works with built-in mathematical text. `tex=True` requires a functioning external LaTeX installation. Share one theme across figures in a study for consistent labels and visual scale.

Grouped comparison bars

Use canonical case reports, not copied sigma arrays or hardcoded coefficient indices:

```python
from basco.plotting import uncertainty_comparison_plot

changed = rows.covariance(noise=bc.DiagonalNoise(2.0 * rows.sigmas))
cases = {
    "Baseline": bc.make_presentation(
        result, reports={"mass": result.mass_properties()},
    ),
    "Twice the measurement noise": bc.make_presentation(
        changed, reports={"mass": changed.mass_properties()},
    ),
}
comparison = uncertainty_comparison_plot(
    cases,
    table="mass",
    metric="relative_percent",
    labels=("primary.mass", "secondary.mass"),
    identifier="mass_noise_trade",
    title="Individual mass uncertainty",
)
bc.write_presentation(
    cases["Baseline"],
    "mass-noise-comparison",
    figures=True,
    plots=(comparison,),
    theme=bc.PlotTheme(tex=False),
)
```

Available metrics are `"sigma"`, `"relative_percent"` and `"gain"`; gain requires an explicit reference presentation. Report IDs, quantity units and conventions must be compatible. Unsupported quantities stay visibly unavailable.

For a new figure, PlotDefinition supplies a named `draw(dataset, theme)` callback returning a Matplotlib figure or `None`, plus a caption. Pass it through the same writer. See extensions and the filesystem contract.

CHAPTER 10

Reuse and trade studies

Propagation is usually the expensive part. Retain a LinearizedStudy and its Transport so changes to the statistical question do not accidentally rerun the dynamics. Identity checks reject reuse when physical inputs or numerical policies no longer match.

The fragments below assume `study`, `rows`, `result` and `bc` from the quickstart.

Reweight or change priors

```
data_only = rows.covariance(priors=())
less_precise = rows.covariance(noise=bc.DiagonalNoise(2.0 * rows.sigmas))
joint = rows.covariance(solver=bc.SquareRootSolver())
```

Each call uses the retained unweighted physical rows and performs **zero new dynamics evaluations**. To replace priors, pass the intended complete producer set with `priors=(...)`. Replacing the solver can change the declared support policy, so retain its diagnostics in comparisons.

Select fewer estimated directions

For source rows containing masses, degree-2 moments, degree-3/4 harmonics and initial-state errors:

```
degree2_parameters = (
    bc.Parameters(rows.parameters.model)
    .estimate_masses()
    .estimate_moments(degrees=(2,))
    .estimate_initial_state()
)
degree2_rows = rows.with_parameters(degree2_parameters, priors=())
degree2_result = degree2_rows.covariance()
```

The original nominal gravity remains order 6. The dropped higher-degree quantities are **fixed at their nominal values**. This is conditioning; marginalization would instead retain their uncertainty and correlations in the solve. Local observation nuisance remains free.

The subset must use the same nominal model, physical directions and initial-state treatment. Reordering compatible existing columns is allowed; adding directions or changing the raw/harmonic map is not an arbitrary view operation. Priors reset intentionally, because an old prior may not define the new statistical problem.

Change channels on retained epochs

```
position_plan = bc.ObservationPlan.uniform(
    stop=study.observations.duration,
    cadence=study.observations.cadence,
    channels=(bc.RelativePosition(sigma=1.0),),
)
position_study = bc.CovarianceStudy(
    study.model,
    study.parameters,
    position_plan,
    integrator=study.integrator,
)
position_rows = position_study.linearize(transport=rows.transport)
position_result = position_rows.covariance()
```

This reassembles observation rows from compatible retained transport. A blackout or per-stream schedule on the same grid can follow the same route. There are no ODE evaluations during the reassembly, but the channels' value/partial maps still evaluate.

If the source study explicitly used native transport, the new study must also specify `transport=bc.NativeAnalyticTransport()`. Defaults are AD; provider identity is checked rather than silently relabeling native arrays as AD.

Change nominal gravity fidelity

Order-4 and order-6 dynamics are different forward models and require separate propagation. For each, construct a `BinaryDynamics` using the same declared system/state and its intended `HouGravity(mutual_order=...)`, then build its own `Parameters` attached to that model.

Hold duration, integration accuracy, observations, priors, parameter meaning and reporting conventions fixed when the goal is to isolate force-model order. A degree-4 estimate under order-4 versus order-6 gravity is a valid pair of model-conditioned covariance studies. The difference between their sigmas is not a direct estimate of truncation bias.

The Hera degree-trade runner separates fresh `propagate` runs from zero-dynamics `compare` runs. Its explicit `--derivatives ad|native` option keeps retained-provider identity clear. The validated degree trades explain the paper and alternative experiments.

Reuse rules at a glance

CHANGED INPUT	REUSE RETAINED ROWS?	REUSE EXISTING TRANSPORT?
Noise, priors, covariance solver or threshold	Yes	Already retained
Compatible subset/reordering of existing solve-fors	`with_parameters(...)`	Already retained
Channels, sigma defaults or mask on same grid	Reassemble rows	Yes, if identity matches
Gravity order, body moments/masses or initial state	No	No
Integrator step/policy or derivative provider	No	No
New required epochs or new physical directions	No arbitrary reuse	Requires appropriate compatible transport

The transport identity includes physical data, coordinate maps, initial state, epochs, provider and integration policy. The linearization identity also captures row assembly. The final analysis identity adds weighting, priors, scales and numerical support choices while excluding wall-clock timings.

Save evidence with the comparison

For each case, export the computed result, its configuration and provenance. Mark whether it was fresh propagation, archived-row replay or new analysis of retained BASCO transport. Then use canonical comparison bars to ensure labels, conventions and uncertainty statuses are checked across cases.

The reuse example is a small runnable starting point. For independent arcs, multi-arc reuse retains one checked transport per named arc.

CHAPTER 11

Shape ensembles and Kaula priors

BASCO separates geometry, shape sampling, coefficient statistics, prior construction and plotting. A shape campaign does not automatically change the gravity model or estimated degree. A Kaula-style prior is an empirical statistical assumption derived from an explicitly defined ensemble.

Create native-ready body moments from an OBJ

```
import basco as bc

mesh = bc.TriangleMesh.from_obj("primary.obj", scale_to_m=1.0)
distribution = mesh.to_distribution(
    mass_kg=5.2e11,
    max_degree=6,
    reference_radius_m=390.0,
)
primary = bc.Body("primary", distribution)
```

The file path must exist. `scale_to_m=1000.0` is appropriate for a kilometre-based OBJ. TriangleMesh requires a closed, consistently outward-wound triangular surface. Checks cover topology, degenerate faces and positive signed volume; they do not certify arbitrary self-intersection freedom.

`to_distribution()` recenters and principal-aligns the mesh, integrates mass-weighted moments, checks structural residuals and clears only permitted roundoff. This supplies the centered/principal-axis representation required by both Hou routes. The fixed harmonic radius and enclosing bound remain distinct.

For a self-contained generated geometry:

```
mesh = bc.TriangleMesh.ellipsoid([400.0, 350.0, 300.0], subdivisions=2)
distribution = mesh.to_distribution(mass_kg=5.2e11, max_degree=6)
```

This is a finite-resolution triangulation. `UniformEllipsoid` gives analytic moments and should not be expected to agree exactly with every coarse mesh.

Volume moments and mass moments

`mesh.moments(max_degree=4, mass_kg=...)` produces mass-weighted moments in kg m^n without automatically changing the frame. Omitting `mass_kg` produces geometric volume moments in m^{n+3}. Those geometric moments cannot be passed as if they already included density.

`mesh.aligned()` returns center-of-mass/principal-axis geometry. The frame convention matters both for body construction and for whether perturbed ensemble members are realigned.

A small fresh shape campaign

The following continues from the generated mesh/distribution above. It creates four perturbed copies as a compact API demonstration, not a statistically mature prior:

```
sampler = bc.RadialShapeSampler(
    roughness_m=2.0,
    correlation_lengths_m=(50.0, 100.0),
    copies_per_length=2,
    seed=17,
)
ensemble = bc.build_shape_ensemble(
    mesh.aligned(),
    sampler,
    reference_radius_m=distribution.reference_radius,
    body="primary",
    max_degree=4,
    density_models=(),
    realign=True,
)
rule = bc.KaulaRule.fit(
    ensemble,
    fit_degrees=(2, 3, 4),
    apply_degrees=(2, 3, 4),
    lock_exponent=None,
)
print(rule.sigma("C20"))
ensemble.to_jsonl("my-primary-ensemble.jsonl")
```

RadialShapeSampler uses seeded spherical-harmonic radial fields with degree 0/1 removed and rescaled to requested RMS roughness. The default seed convention distinguishes the actual requested correlation lengths. `legacy_integer_length_seed=True` explicitly selects the integer-length seed convention used for paper reproduction.

`realign=True` recenters/principal-aligns each sampled shape. `realign=False` retains the paper's perturbation-frame convention. This choice changes the statistics and must remain in provenance.

Resolution, correlation lengths, roughness, sample count and fitted degree range are scientific campaign settings. Large high-resolution degree-12 campaigns can be expensive; sampling degree 12 does not enable degree-12 Hou propagation or independent solve-fors.

Optional interior variations

DensityVariation offers mass-preserving `"core"`, `"shell"` and `"torus"` alternatives, with contrast and region dimensions. Supply them through `density_models=(...)`:

```
interiors = (
    bc.DensityVariation("core", contrast=0.2),
    bc.DensityVariation("shell", contrast=-0.2),
)
```

This defines alternatives only; pass `interiors` to a new `build_shape_ensemble(...)` call to use them. Check region containment in the actual body. A generic mesh check does not prove an arbitrary core or torus lies wholly inside it.

What is fitted?

KaulaRule fits independent zonal and nonzonal coefficient-magnitude power laws. The shape-member RMS informs the fit, and adopted upper amplitudes cover retained shape/interior members. These magnitudes are not a cloud of nominal-subtracted estimation residuals.

`fit_degrees` selects the degrees used for fitting; `apply_degrees` selects degrees receiving prior sigmas. `lock_exponent=2.0` adopts that exponent only when it lies within the fitted one-sigma interval. It does not force every fitted law to follow degree^{-2}.

The ensemble and rules use **geodesy 4-pi normalized**, dimensionless coefficients at an explicit fixed radius. Ordinary `result.harmonics()` reports **unnormalized** C/S. The prior producer performs the conversion; do not manually paste normalized sigmas into an unnormalized covariance.

Apply the rule through the normal prior interface

After composing `model`, `parameters` and `measurements` for the corresponding body:

```
shape_prior = bc.KaulaPrior(
    parameters,
    {"primary": rule},
    raw_moment_priors=True,
)
study = bc.CovarianceStudy(
    model,
    parameters,
    measurements,
    integrator=bc.RK4(dt=60.0),
    priors=(shape_prior,),
)
```

The rule radius must match the model radius. Body keys may be `primary` / `secondary` or `A` / `B`. A rule for one body adds no prior to the other. No mass, initial-state or barycenter prior is implied.

`raw_moment_priors=True` adds the paper's separate independent raw-moment constraints using RMS touching-coefficient sigmas. It is a modeling choice, not an inverse transform of harmonic covariance. Set it false for harmonic knowledge alone and retain any resulting trace/inertia null directions honestly.

Persist and plot

```
loaded = bc.CoefficientEnsemble.from_jsonl("my-primary-ensemble.jsonl")
saved_rule = rule.to_dict()
reloaded_rule = bc.KaulaRule.from_dict(saved_rule)

from basco.kaula_plotting import render_kaula
render_kaula(
    {"primary": rule},
    "my-kaula-figures",
    theme=bc.PlotTheme(tex=False),
)
```

The plotting call needs a rule retaining its ensemble and a new output path. It renders existing fitted data; it performs no propagation or fresh sampling. Older JSONL files without metadata require explicit `body` and `reference_radius_m` when loaded.

The Hera Kaula example separates replay/refitting of captured paper clouds from `--fresh` sampling. Label those activities accurately. The pinned benchmark prior remains available for exact historical reproduction; the generic components above support a new binary's own campaign.

CHAPTER 12

Independent arcs and estimation

Multi-arc analysis estimates shared physical quantities from several locally parameterized trajectories. Masses and gravity can be common while each arc retains its own initial state and observation nuisance. BASCO supports both a convenient continuous-nominal timeline and direct assembly of independent arc linearizations.

A continuous nominal, independently estimated arc states

This fragment assumes `model`, `parameters`, `channels` and `bc` from your study. Include `.estimate_initial_state()` in `parameters` when arc-start binary states are to be coestimated:

```
plan = bc.ObservationPlan.uniform(
    stop=14 * 86400.0,
    cadence=900.0,
    channels=channels,
    schedule=bc.PeriodicBlackout(period=86400.0, duration=8 * 3600.0),
)
timeline = bc.ArcTimeline.every(3.5 * 86400.0, stop=14 * 86400.0)
campaign = bc.MultiArcCovarianceStudy(
    model,
    parameters,
    plan,
    timeline,
    integrator=bc.RK4(dt=75.0),
)
multi = campaign.linearize()
result = multi.covariance()
print(result.summary())
print(campaign.arc_summary(multi))
```

ArcTimeline cuts contiguous mission intervals starting at zero. `every(...)` creates equal lengths and a shorter final remainder when needed; `from_bounds(...)` accepts explicit bounds/names. Arc lengths must be compatible with the plan cadence and the integration grid. A boundary observation belongs to the arc ending there.

MultiArcCovarianceStudy propagates each arc from the preceding nominal end state, with an explicit SO(3) projection at the restart. Schedules evaluate mission time, so daily gaps remain daily across boundaries. It defaults to Hou AD; pass `transport=bc.NativeAnalyticTransport()` for manual derivatives.

Nominal continuity does **not** impose estimation continuity constraints. Local state errors remain independently parameterized; no process noise or sequential-filter assumption is introduced.

Reuse the arc transports

```
retained = campaign.transports(multi)
```

Construct another campaign with the same physics, timeline, state treatment and output grid but different observation schedules/channels, then call:

```
changed_multi = changed_campaign.linearize(transports=retained)
changed_result = changed_multi.covariance()
```

Here `changed_campaign` is the newly configured `MultiArcCovarianceStudy`. Each named arc's transport is checked. Noise/prior changes on `multi` require neither this reassembly nor propagation.

The multi-arc example compares a single arc, gaps and independent arcs, then sweeps blackout duration with retained transport.

Assemble independent linearized studies directly

When arcs have independently supplied start states, build each ordinary study first. For `early_rows` and `late_rows` returned by those studies:

```
multi = bc.MultiArcStudy(
    {
        "early": early_rows.with_priors(()),
        "late": late_rows.with_priors(()),
    },
    priors=shared_priors,
    solver=bc.NuisanceProjectedSolver(),
)
result = multi.covariance()
```

`shared_priors` is the tuple of global prior producers you intend to apply once. Using `with_priors(())` avoids carrying a shared prior separately from each arc. Genuine local priors may remain on their arcs; any arc prior touching a shared parameter is rejected to prevent double counting.

By default every physical parameter ID must agree between arcs and is shared. Explicit `shared_parameters=("primary.mass", ...)` shares a subset; other physical coordinates become local. Shared nominal values, units and physical meaning must match. Local labels become `arc.early.<source-label>` and `arc.late.<source-label>`; shared IDs retain their names.

`multi.arc_columns(name)` maps a source arc's coordinate order into global columns. The full supported covariance retains cross-arc and local/shared cross blocks. A complete common physical report needs a complete shared physical map; partial sharing may require named covariance blocks or a custom derived transform.

Noise across arcs

Independent per-arc noise is the default. Pass a mapping such as `noise={"early": early_noise, "late": late_noise}` to replace weights per arc. A single full correlated model can represent cross-arc observation correlation, following the assembled scalar row order.

Shared prior knowledge enters once globally. Global correlated priors can also use fully qualified local coordinate IDs when the statistical model actually connects those local quantities.

Solve a frozen-linear correction

```
fit = multi.estimate({
    "early": observed_minus_predicted_early,
    "late": observed_minus_predicted_late,
})
print(fit.corrections_for(("primary.mass", "secondary.mass")))
print(fit.covariance.summary())
```

The two vectors are externally supplied **observed minus nominal prediction** residuals, in the exact source arc row order and physical units. This is one weighted linear correction at a fixed nominal, including prior mean-minus-reference residuals. It is not a nonlinear mission-data fit.

MultiArcEstimate retains corrections, covariance, postfit residuals and diagnostics. A deficient solve identifies supported combinations and records its minimum-scaled-norm convention for unresolved correction components; unresolved values are not thereby physically determined.

Nonlinear batch/MAP callbacks

MultiArcBatchEstimator supplies a generic relinearizing Gauss–Newton driver with actual-objective backtracking:

```
solution = bc.MultiArcBatchEstimator(
    max_iterations=15,
    step_rtol=1e-9,
    gradient_rtol=1e-9,
).solve(
    initial_guess,
    linearize=build_arcs_and_residuals,
    retract=apply_named_correction,
)
print(solution.converged, solution.status)
```

This is an interface fragment: you must implement both callbacks for your application. `build_arcs_and_residuals(state)` returns `(MultiArcStudy, residual_mapping)` at that nominal. It rebuilds predictions, rows and prior references while keeping observations, noise and prior means fixed.

`apply_named_correction(state, correction, layout)` returns a new state consistent with the rows' named differential. It owns finite attitude retraction and positive-mass handling. Neither callback may mutate its input.

Convergence, exhausted iterations, rank deficiency and failed line search remain distinct statuses. Returned covariance belongs to the returned statistical nominal; backtracking adds no fictitious information.

Current capability boundary

Generic nonlinear numerical tests exist. Complete binary observed-value ingestion, measured-attitude residual adapters and seeded mission-style recovery are unfinished. Sequential filters/process noise are planned, not implemented interchangeable filter choices. Do not describe a covariance reproduction or a callback API as a validated real Hera-data fit.

The multi-arc design and schedule/arc evidence record the tested semantics and limitations.

CHAPTER 13

AD, analytic derivatives and validation

BASCO preserves the manual analytic Hou implementation and provides an independent JAX-compatible value implementation differentiated automatically. **Hou AD is the default.** Finite differences are an explicit diagnostic, not part of normal covariance runs.

Choose the route in the study

For `model`, `parameters`, `plan` and `bc` from your ordinary composition:

```
# Default: automatic differentiation of the copied Hou equations.
ad_study = bc.CovarianceStudy(model, parameters, plan, integrator=bc.RK4(60.0))

# Explicit manual analytic route, without JAX.
native_study = bc.CovarianceStudy(
    model, parameters, plan,
    integrator=bc.RK4(60.0),
    transport=bc.NativeAnalyticTransport(),
)
```

Explicit AD selection is also available:

```
from basco.ad import ADTransport
ad_study = bc.CovarianceStudy(model, parameters, plan, transport=ADTransport())
```

The managed plain `run` environment includes AD; `run --native` is analytic-only; `run --plot` includes AD and plotting. An environment flag does not change a provider explicitly chosen inside a script.

Importing BASCO, constructing `ADTransport` and describing a study do not load JAX. Propagation loads the optional dependency. If JAX is unavailable, the error asks for the AD extra or explicit native selection. There is no implicit fallback.

What automatic differentiation changes

AD evaluates exact chain-rule derivatives of the operations in the implemented floating-point value function. It avoids manually maintaining every algebraic partial, and does not subtract perturbed forward values as finite differences do. It is still subject to the model's equations, conventions, floating-point arithmetic and integration accuracy.

BASCO's AD evaluator computes force/torque derivatives and the dynamics `F`, `A = dF/dx` and raw-moment `B`. The shared augmented fixed-RK4 integrator propagates the same variational structure. The provider converts raw sensitivities into the declared SI moment units before returning transport.

This is AD of the implemented continuous right-hand side integrated through variational equations. It is not a blanket differentiation of an entire user runner, file I/O, plotting, rank-threshold decisions or arbitrary custom observations. Existing physical-coordinate maps and observation partials remain important explicit links.

The maintained domain is centered/principal-axis rigid, autonomous Hou orders 2/4/6, fixed RK4 and uniform positive output epochs. Arbitrary deformation, collisions, off-grid observation epochs and new forces do not become supported merely because JAX can differentiate Python operations.

Find the equations in the source

The mathematical chain is deliberately split into recognizable owners:

EQUATION OR MAP	SOURCE TO READ
Retained Hou potential, moment rotation and native `hou_ode`	Native GUBAS core
Hand-derived evaluator and prepared derivative access	analytic.py, kernels.py
Copied Hou contraction tables and potential	ad/hou_tables.py, ad/gravity.py
Copied equations of motion and AD evaluator	ad/dynamics.py, ad/evaluator.py
Shared variational RK4 stages	integration.py
Independent parameters to raw moments; physical state tangents	coordinates.py, state.py
Observation/direct mass terms and row assembly	observations.py, linearization.py
Exterior coefficient and reported-quantity maps	harmonics.py, reports.py

Read these with the math-flow overview, rather than treating the derivative backend as a single opaque Jacobian. The API inventory describes signatures and fields; source links expose the implemented equations themselves.

Compare both methods at matched inputs

```
ad_rows = ad_study.linearize()
native_rows = native_study.linearize()
ad_result = ad_rows.covariance()
native_result = native_rows.covariance()
print(ad_result.mass_properties().summary())
print(native_result.mass_properties().summary())
```

These are two genuine propagations. Compare physics, scales, rows, ranks and reports at the same model/state, epochs, integration step, parameter basis, noise, priors and solver. A timing comparison with different assumptions is not a derivative-backend benchmark.

The AD example runs the native/default-AD Hou comparison. Default/native snapshots and the fresh four-case Hera rerun passed the Phase 2 acceptance. The paper's 36 printed Table 2 values reproduced through default AD. This is evidence for the tested configurations, not universal certification of all future extensions.

Run the built-in finite-difference checks

From the checkout:

```
python3 -B tools/dev.py validate
python3 -B tools/dev.py ad_validate
```

`validate` selects native FD checks; `ad_validate` selects AD checks. Ordinary `test`, covariance and export commands do not launch FD probes. Existing validation covers selected local dynamics, raw/physical maps, observations and reports. Complete all-chain/companion-covariance coverage remains unfinished; read each record's requested directions and settings.

A small independent derivative check

The following standalone example demonstrates the public validation engine on a simple nonlinear function. It does not validate binary dynamics:

```python
import numpy as np
from basco.validation import ProbeRequest, Tolerances, validate_jacobian

def forward(x):
    return np.array([x[0]**2, x[0] * x[1]])

def jacobian(x):
    return np.array([[2.0 * x[0], 0.0], [x[1], x[0]]])

request = ProbeRequest(
    point=np.array([2.0, 3.0]),
    input_scales=np.ones(2),
    output_scales=np.ones(2),
    steps=(1e-2, 1e-3, 1e-4, 1e-5, 1e-6),
    input_ids=("x", "y"),
    output_ids=("x_squared", "x_times_y"),
    provenance={"purpose": "validation-engine tutorial"},
)
print(request.preview())
check = validate_jacobian(
    forward,
    jacobian,
    request,
    Tolerances(atol=1e-8, rtol=1e-8, fd_atol=1e-7, fd_rtol=1e-7),
    name="two_variable_example",
    equation_id="tutorial.polynomial",
    provider="manual_analytic",
)
print(check.summary())
check.write("validation/two-variable-example")
```

ProbeRequest declares physical point, positive input/output scales, decreasing dimensionless step sizes, directions and named axes. Optional `directions` has shape `(input coordinates, selected directions)`; probing a subset does not certify the rest. `preview()` estimates forward-call cost before evaluation.

Tolerances separately controls analytic agreement and FD-reference stability. The engine chooses a reference step from agreement among FD samples, independently of the proposed analytic derivative. Forward roundoff limits, failed evaluations and unrepresentable perturbations are recorded. `pass`, `mismatch` and `inconclusive` are different outcomes.

Validate physical directions

For binary attitude, supply a retraction inside the forward adapter and perturb physical SO(3) tangents. Do not independently perturb nine matrix entries. For masses and moments, update the entire declared physical map, including coupled dimensional moments. Use scales even at zero nominal coefficients.

DerivativeRegistry and JacobianCheck organize explicitly selected checks with equation IDs, and report unrequested coverage. There is no hidden global import-time registry.

FD artifacts save points/scales, sampled derivative actions, selected steps, statuses and provenance without overwriting an existing baseline. Derivative checks complement integration-step refinement, conservation and reference comparisons; none substitutes for all the others.

See AUTODIFF, derivative-validation design and the validation source for detailed contracts.

CHAPTER 14

Extend one component at a time

BASCO uses composition and structural contracts. A compatible object implements a small interface rather than inheriting a large application framework. Replace the owner of the behavior you need; keep units, frames, coordinate IDs and identity settings explicit at that boundary.

Contract map

NEW CAPABILITY	CONTRACT	WHAT IT MUST OWN
Measurement	ObservationChannel	Labels, units, positive sigmas, state and direct physical prediction partials
Timing/visibility	ObservationSchedule	Boolean mask on existing epochs and complete JSON description
Alternative gravity declaration	GravityModel	Mutual order and optional compatibility/identity settings
Custom AD potential	DifferentiableGravityModel	JAX-traceable SI mutual potential
One variational evaluation	VariationalEvaluator	Callable returning native-layout `F, A, B`
Propagation backend	TransportProvider	Capabilities, preparation, integration and checked transport identity
Measurement statistics	NoiseModel	Consistent vector/matrix whitening and complete settings
Physical prior	PriorProvider	Prior rows, mean/reference, layouts, noise and provenance
Covariance strategy	CovarianceSolver	Support policy, covariance/null/cross information and diagnostics
Linear correction	LeastSquaresSolver	A correction using the covariance's same support decision
Figure	PlotDefinition	A canonical-data draw function and caption

Protocols are runtime-checkable, but `isinstance(...)` only confirms structural member presence. The consumer still validates returned shapes, finite values, units and identities. A successful structural check is not scientific conformance evidence.

Add a covariance range channel

This example reuses the existing relative-position derivative and applies a line-of-sight chain rule. It is a covariance prediction-direction channel, not a complete observed-range data adapter:

```
import numpy as np
import basco as bc

class RelativeRange:
    labels = ("relative_range",)
    units = ("m",)
    sigma = 1.0

    def linearize_native(self, x, parameters):
        position_rows, direct = bc.RelativePosition().linearize_native(x, parameters)
        rho = 1000.0 * (x[12:21].reshape(3, 3) @ x[:3])
        line = rho / np.linalg.norm(rho)
        return line[None, :] @ position_rows, line[None, :] @ direct

plan = bc.ObservationPlan.uniform(
    stop=3600.0,
    cadence=600.0,
    channels=(RelativeRange(),),
)
```

The channel returns `(dh/dnative30, direct dh/dphysical_parameters)`. The native matrix and kilometre conversion appear here because this low-level observation adapter explicitly consumes native30. Ordinary runners should continue using physical classes rather than copying native indices elsewhere.

Declare known zero derivatives as zero arrays of the correct shape. Do not omit a derivative and hope it will be interpreted as zero. Validate the new channel independently, then use it with the ordinary study and either Hou transport.

The dynamics AD default does not automatically differentiate this custom observation. General custom-observable AD remains unfinished.

Optional observation capabilities

`ObservationChannel` requires only labels, units, sigma and `linearize_native`. Separate optional contracts in contracts.py describe:

OPTIONAL CONTRACT	ADDED CAPABILITY
`ScheduledObservation`	A `schedule` selecting existing epochs
`NuisanceObservation`	Independent nuisance layout and `nuisance_partials(epoch)`
`ValueObservation`	An absolute `value_native(...)` prediction

A covariance-only channel need not claim absolute values. Conversely, a value predictor alone does not provide covariance partials. PolynomialBias and PrimaryBarycentricPosition show how nuisance and direct mass dependence remain modular.

Custom gravity and the maintained Hou path

The recommended model is `HouGravity(mutual_order=2/4/6)` with its dedicated native or AD evaluator. It is a gravity declaration whose adapters own the equations; it need not implement a custom-potential method.

A custom differentiable model instead supplies:

```
def potential(self, r, C_AB, T_A, T_B, G):
    ...  # Return the JAX-traceable mutual potential in joules.
```

The contract inputs are SI: relative position in primary components, secondary-to-primary rotation, raw mass-weighted moment tables and G. Declare every model setting affecting physics through identity settings. `moment_solve_fors=False` explicitly refuses unsupported independent moment/harmonic directions. `check_model(model)` can validate body compatibility.

Such a model requires an explicit compatible AD provider; it is never silently run through native Hou. Cartesian multipole is deprecated compatibility, and mascons/second-degree example models are retained reference utilities outside active Hou development. See gravity-model evidence before treating a retained alternative as a maintained general workflow.

Replace the evaluator or provider

The common variational integrator requires a callable returning shapes `(30,)`, `(30, 30)` and `(30, n_raw_slots)`. Its `B` follows the declared raw-slot order and native kg km^n moment units; transport owns conversion to the public raw SI derivative units. Do not mix a native state Jacobian with an SI moment Jacobian accidentally.

A full provider returns Transport with matching epochs, state order, raw slots, physical/raw sensitivities, diagnostics and TransportIdentity. It does not choose observations, priors or solver. Reuse must validate identity rather than trusting an array's shape or filename.

Results and presentation stay separate

Numerical solvers consume named rows/layouts and do not import body models or physical reporting maps. Resolved parameters provide the lazy physical-report service. A generic numerical result can therefore remain useful without pretending it has a binary mass map.

Presentation consumes result/report data and never recomputes gravity or covariance. To add a figure, return a Matplotlib figure from a `PlotDefinition.draw(dataset, theme)` callback and let the ordinary writer handle format selection, files and captions. Preserve row statuses instead of replacing unsupported uncertainty by zero bars.

A small extension checklist

Before calling an extension complete, record its owner and input/output layouts; provide units, frame and epoch conventions; declare every physics/statistics setting in identity/provenance; validate shapes and unavailable capabilities; compare an independent small case or FD action; and demonstrate it in an ordinary readable runner.

Avoid a new global registry or application singleton when an explicit object can carry the behavior. Keep code with the model, observation, statistical or presentation owner that gives it scientific meaning. Troubleshooting covers the common boundary failures.

CHAPTER 15

Example gallery

Every script below is an existing user example in `examples/`. They run against the installed library and compose the same modular components used for custom studies. This gallery documents their real behavior; opening these pages runs no scientific computation.

Choose your first example

GOAL	START HERE	FRESH DYNAMICS?	OUTPUT
Check installation cheaply	So smoke run	One short prefix	Console; optional numeric artifact
Build your own binary	Custom binary	One short prefix	Null-aware console summary
Understand component reuse	Noise/prior/channel reuse	Initial propagation only	Compared summaries
Reproduce the paper	One Hera case	Full 16 orbits	Standard results and figures
Reproduce Table 2	All Hera cases	Four independent full propagations	Table 2 and per-case runs
Check analytic versus AD	Derivative comparison	Both routes	Asserted report agreement
Fit Kaula laws	Kaula and new shapes	No trajectory dynamics	Laws, figures, tables, manifest
Compare degrees/priors	Degree and gravity trades	`propagate` : yes; `compare` : no	Study reports and plots
Assess tracking gaps/arcs	Multi-arc	Continuous and arc transports; sweep reused	Console; optional artifact
Explore old model comparisons	Legacy gravity	Several two-orbit runs	Printed comparison

Running examples reliably

Quick start explains `bootstrap`, `build`, `ad_environment` and `plot_environment`. Use the managed launcher from the repository root:

```
python3 -B tools/dev.py run examples/custom_binary.py
python3 -B tools/dev.py run --ad examples/ad_covariance.py
python3 -B tools/dev.py run --plot examples/hera_kaula.py
```

Plain `run` and `--ad` select the JAX environment. `--plot` selects JAX plus plotting. `--native` selects the analytic-only environment. **The environment flag does not change the transport selected inside your script.** A script that omits `transport` still requests default AD; without JAX it raises an actionable error. For native-only execution the script must pass `bc.NativeAnalyticTransport()` or expose its own corresponding option.

The launcher changes the working directory to the machine cache. Resolve data relative to `__file__` or pass absolute input/output paths, for example `--output "$PWD/results/my-new-run"`. Existing scripts already resolve their own bundled inputs. Numeric artifact writers refuse an existing target; standard exports create descriptive unique run folders. Add `--tex` only when a working external LaTeX installation is available.

Historical timings below are observations from an Apple M4 Pro, not performance guarantees. AD compilation, active CPU jobs and selected gravity degree matter. No examples were rerun just to build this site.

So smoke run

Source: so_covariance.py · Reference factories

```
python3 -B tools/dev.py example
python3 -B tools/dev.py run examples/s0_covariance.py --prefix
python3 -B tools/dev.py run --native examples/s0_covariance.py \
  --prefix --derivatives manual --output "$PWD/results/s0-native-prefix"
```

The synthetic S0 study exercises a rigid binary, degree-2 solve-fors, initial-state errors, position and attitude observations. `--prefix` integrates only one 600-second output interval and costs seconds. The deliberately small observation set leaves null directions; unavailable formal uncertainty is the correct result.

Without `--prefix` the script runs the full configured S0 study. `--dt` changes RK4 seconds; `--derivatives ad|manual` selects AD or hand-derived native partials. `--output` saves a **numeric artifact**, not the full standard figure gallery: epochs, native states, transition/sensitivity matrices, rows, layouts, singular/null information, covariance when available, reports and source hashes.

Custom binary

Source: custom_binary.py · Bodies and states

```
python3 -B tools/dev.py run examples/custom_binary.py
```

This is the smallest fully explicit body/state construction: two SI ellipsoids, a 2 km separation, a near-circular velocity scale, spin vectors and identity rotations. It selects masses, moments and initial-state errors, then requests one 600-second observation epoch with an RK4 step of 60 s. Astropy quantities make unit intent visible. Default AD is required; no plotting is needed. It prints unresolved directions and writes no results folder.

Use it as a starting point for your own script, but lengthen the study and validate its dynamics before interpreting scientific uncertainties. A geometrically convenient initial orbit is not a full-field equilibrium solve.

Reuse noise, priors and channels

Source: reuse_covariance.py · Reuse boundaries

```
python3 -B tools/dev.py reuse
```

A 1,200-second S0 prefix is propagated once. The script compares data-only covariance, an independent 5% primary-mass prior, doubled measurement noise and position-only channels. The new prior does not change the information carried by the data. The new noise and prior need no ODE evaluations; the compatible channel change reuses the checked transport. Output is printed summaries and a mass-properties report. Cost is seconds plus first AD compilation.

One Hera paper case

Source: hera_covariance.py · Complete annotated study

```
python3 -B tools/dev.py hera
```

The default is full tumbling (`AC3`), 16 mutual orbits, gravity order 6, both masses and raw degree-2/3/4 moments, free barycenter offset/rate, and the paper's combined Kaula prior. The script prints the paper-normalized report and writes a standard results folder including `paper` , `harmonics` and `mass` tables and PNG/SVG/PDF figures. It requests external LaTeX. The managed plotting environment includes JAX; there is no native fallback.

Historical full-case timings: about 1–2 minutes AD versus about 25 minutes native; Phase 2's concurrent AD acceptance took 199–201 seconds per case. To change the study, modify the readable `make_study(...)` call in your own runner. Available factory arguments are documented in the Hera chapter.

All four Hera cases

Source: hera_all_cases.py

```
python3 -B tools/dev.py hera_cases
python3 -B tools/dev.py hera_cases AC1 AC3 --jobs 1
python3 -B tools/dev.py run examples/hera_all_cases.py \
  AC3 --orbits 2 --no-export
```

The four cases share one factory and differ in initial rotation. Independent processes propagate each case, print progress and compare the nine selected uncertainty percentages to Table 2's printed rounding. With all cases the target is **36/36**. Default exports write one standard run per case; `--no-export` prints only; `--figures` adds plots when launched with `--plot`. Other options are `--orbits`, `--dt`, `--jobs` and `--output-root`.

The two-orbit command is a diagnostic prefix; its percentages are not expected to reproduce the 16-orbit table. A numerical conditioning refusal is reported, not patched by silently weakening the solver.

Hou native versus AD

Source: ad_covariance.py · Derivative methods

```
python3 -B tools/dev.py ad_example
```

The full S0 study runs twice with the same model, parameter selection, observation plan and RK4 settings. One provider is `NativeAnalyticTransport`; the other is the default Hou AD route. The script asserts equal posterior rank and mass/degree-2 sigma agreement at `rtol=1e-8`. Phase 2 observed a maximum mass/GM/ratio sigma difference of 5.5e-12 relative. It prints the checks and mass report; it writes no artifact. An earlier measured execution took about 35 seconds.

This is a useful same-problem backend comparison. It does not replace opt-in finite-difference checks or certify all nonlinear inference behavior.

Kaula laws and new shapes

Source: hera_kaula.py · Shapes and Kaula

```
# Refit the paper's retained coefficient clouds, no new meshes or trajectories.
python3 -B tools/dev.py run --plot examples/hera_kaula.py --tex

# Generate a small, new, seeded low-resolution shape/density ensemble.
python3 -B tools/dev.py run --plot examples/hera_kaula.py --fresh \
  --max-degree 4 --copies 3 --subdivisions 1
```

Archived mode verifies source cloud hashes, fits separate zonal/nonzonal power laws and checks all adopted law parameters/sigmas against the paper. It reproduces Figure 3 in roughly 10 seconds historically. The default input roots are BASCO's captured paper/Kaula ensembles; `--archive` and `--ensembles` replace them.

Fresh mode calls the generic mesh, radial sampler and core/shell/torus density components with a fixed seed. The CLI permits `--max-degree` 3–12, `--copies` per correlation length and `--subdivisions` for ellipsoid resolution. This is preprocessing, not trajectory integration. Cost grows with degree, mesh resolution and ensemble size. The demonstration is not a claim to reconstruct the full original 200-member random ensemble.

Outputs include PNG/SVG/PDF figures, `tables/kaula_laws.csv`, `tables/kaula_envelopes.csv`, `results.json`, `validation.json`, `README.md` and a byte manifest. `--output` selects a new destination; `--tex` is optional.

Degree, prior and gravity trades

Source: hera_degree_trades.py · Hera trade interpretation

```
python3 -B tools/dev.py run --plot examples/hera_degree_trades.py --help

# A fresh active-gravity-4 transport, default AD.
python3 -B tools/dev.py run --ad examples/hera_degree_trades.py propagate \
  --gravity-degree 4 --output "$PWD/results/hera-ac3-g4-ad-transport"

# Existing retained native transports are a different, explicit provider.
python3 -B tools/dev.py run --plot examples/hera_degree_trades.py compare \
  --derivatives native \
  --transport-root "$PWD/artifacts/scientific" \
  --output "$PWD/results/hera-degree-comparison" --figures
```

`propagate` accepts `--state`, `--gravity-degree`, `--dt`, `--duration`, required `--output` and `--derivatives ad|native`. It stores a fresh complete numeric transport artifact with source identity.

`compare` assembles six paper Figure 5 cases and additional symmetric degree/prior experiments from **completed BASCO transports**. It performs no new dynamics. It needs local transport artifacts named `hera-ac0-dt75-v1` through `hera-ac3-dt75-v1`; these large artifacts are ignored by Git and are not guaranteed to exist in a clean clone. The script does not download or fabricate them. `--gravity4-transport` adds a matched order-4 comparison, `--figures` draws the reports, and `--tex` requests LaTeX. Its default provider is native because the retained fixtures are native. All source transports in a comparison must match its declared provider.

The two commands above demonstrate separate activities: the newly produced AD order-4 artifact cannot be substituted into an otherwise native comparison. Choose one backend consistently and create/reuse the matching completed inputs. See the Hera chapter for the precise Figure 5 parameter spaces and null-direction caveats. Historical `compare` cost was about 20 seconds.

Multi-arc and tracking gaps

Source: multi_arc_covariance.py · Multi-arc guide

```
python3 -B tools/dev.py multi_arc
python3 -B tools/dev.py run examples/multi_arc_covariance.py --help
python3 -B tools/dev.py run examples/multi_arc_covariance.py \
  --state AC1 --days 7 --arc-days 3.5 --blackout 8
```

Defaults are AC3 full tumbling, Hou order 2, 14 days, 15-minute cadence, 8 h/day off, four 3.5-day arcs, RK4 75 s and AD. Masses/gravity are shared; each arc gets its own binary initial state and channel nuisance coordinates. Order 2 uses no field prior; `--order 4|6` estimates moments through degree 4 with the paper prior. Other options are `--cadence` in minutes, `--dt` in seconds, `--derivatives ad|manual`, and optional numeric-artifact `--output`.

The script compares a continuous ungapped study, the same continuous transport with a blackout, independent arcs, and a blackout sweep. Reweighting/selection in the sweep reuses per-arc transports. It prints a conditioning refusal when a continuous chaotic arc exceeds the solver guard; that refusal is an intended result. Historical default execution was about four minutes on the measured Mac; higher-order longer studies cost more.

These are covariance studies. The separate generic nonlinear batch driver exists, but this example does not fit actual Hera observations or apply a sequential filter.

Legacy gravity comparison

Source: gravity_models.py · Capability status

```
python3 -B tools/dev.py run --ad examples/gravity_models.py
```

This retained reference creates two point-mass lattices, propagates two orbits under exact mascon gravity, Hou orders 2/4/6 and Cartesian multipole orders 3/5, and prints trajectory differences and mass sigmas. It uses AD and selects masses/initial-state errors. It does not produce the Hera paper reproduction.

Cartesian multipole is deprecated compatibility code; the primary supported scientific workflow is Hou gravity. Mascons and second-degree reference utilities remain available but are outside current primary development. This example can be computationally heavier than a short S0 study and has no promised current timing.

Evidence tools are separate from user examples

Scripts under tools/evidence preserve numerical acceptance, archived replay and publication generation. They may require completed local artifacts. They are not all minimal tutorials, and not every one should be run to start a project. Use capabilities to distinguish available features from planned estimation/filter work.

CHAPTER 16

Capabilities and limits

BASCO's maintained focus is **formal local covariance for binary-asteroid masses and gravity**, using the Hou/GUBAS full two-body dynamics. This map describes the completed Phase 2 checkpoint. A class name or a designed interface alone does not establish a validated mission-data pipeline.

Supported means implemented with recorded checks in the stated domain. **Partial** means useful implemented pieces exist but a wider workflow or validation remains open. **Legacy** means retained compatibility/reference functionality. **Planned** means it is not a delivered feature today.

Bodies, gravity and integration

FEATURE	STATUS	SCOPE AND ENTRY POINT
Rigid autonomous binary	Supported	Centered principal-axis body representation; BinarySystem, BinaryDynamics
Uniform ellipsoids	Supported	SI axes/density and nominal moments; UniformEllipsoid
Explicit moment tables	Supported	Complete nominal moments with fixed reference/bounding radii; MomentDistribution
Closed triangular meshes/OBJ	Supported	Geometry validation, centering/principal alignment, moments and distribution construction; TriangleMesh
Seeded shape/density ensembles	Supported	Radial perturbations and core/shell/torus alternatives; RadialShapeSampler, DensityVariation
Hou mutual orders 2, 4, 6	Supported, primary path	Order is independent of nominal input degree and selected solve-fors; HouGravity
Fixed-step RK4 augmented transport	Supported	Native30 storage with consistent state/sensitivity epochs; RK4
Exterior-domain checks	Supported	Available body moments and declared bounding geometry checked; this is not a contact/collision dynamics model
Exact mascon, second-degree reference models	Legacy/reference	Implemented AD utilities retained outside the primary Hou development target
Cartesian multipole gravity	Deprecated legacy	Old imports remain; construction warns; implementation under `basco.ad.legacy`
Independent degree-5/6 solve-fors	Planned	Degree-6 **nominal gravity** works; this does not expose independent uncertain degree-5/6 coordinates

FEATURE	STATUS	SCOPE AND ENTRY POINT
Adaptive integrators, events, external perturbations	Planned	Current public study path is the documented fixed-step autonomous model
Deformation/contact/free non-principal representation	Planned	Not implied by a generic gravity contract

See bodies and states, derivatives and gravity compatibility evidence.

Parameters and physical reports

FEATURE	STATUS	SCOPE AND ENTRY POINT
Both masses	Supported	Positive mass coordinates with full effect on force, inertia and reports; Parameters
Degree-2 normalized moments	Supported	Three diagonal principal-axis moments per body
Raw degree-3/4 moments	Supported	Complete monomial coordinates; trace null directions are retained
Exterior degree-3/4 harmonic coordinates	Supported	Fixed reference radius; fixed trace complement; body-specific selections
18 physical initial-state errors	Supported	SI position, velocity, spins and six local attitude errors; distinct from native30 storage
Explicit coordinate scales/layouts	Supported	Stable labels and units; scales improve numerical conditioning and add no information; CoordinateLayout
Fixed-G mass, GM, total and ratio reports	Supported	Uses full joint covariance; CovarianceResult
Harmonics and dimensional inertia reports	Supported	Report conventions are recorded; paper-normalized report is separate
Custom derived report	Supported	User value/Jacobian with declared labels/units and null-aware transformation
Conditioning a compatible solve-for subset	Supported	Reuses rows while treating omitted solve-fors as fixed, not marginalized uncertainty
Mesh-vertex/density/mascon solve-fors	Planned	Nominal preprocessing or a reference value model does not imply estimable geometry
Estimating G separately from masses	Planned	Current G is fixed in the model

See parameters, results and the glossary for conventions. A reported fixed coefficient is different from an estimated coefficient with informative data.

Measurements and schedules

FEATURE	STATUS	SCOPE AND ENTRY POINT
Inertial relative position	Supported	Three components; RelativePosition
Both body-attitude covariance tangents	Supported	Three local directions per body; BodyAttitude
Primary barycentric position	Supported	Direct mass-ratio partial plus propagated dependence; PrimaryBarycentricPosition
Polynomial observation bias	Supported	Explicit SI offset/rate/higher polynomial terms; PolynomialBias
Uniform positive elapsed epochs	Supported	First observation after one cadence; no invented off-grid final sample
Blackouts, passes, windows/duty cycles	Supported	Masks on the base uniform grid; PeriodicBlackout, Periodic, Windows
Per-stream schedules/cadence masks	Supported	Scheduled, Every, schedule composition with `&`, `|`, `~`
Custom analytic observation channels	Supported extension point	Conform to the observation contract, with explicit value/derivative availability and tests
Arbitrary custom-observable AD	Planned	Default AD differentiates the maintained dynamics; it does not automatically differentiate every user channel
Irregular off-grid observations	Planned	Schedule masks select available grid epochs
State-dependent visibility	Planned	Occultation, field of view and phase-angle geometry are not provided
Real Hera data ingestion/range/Doppler adapters	Planned	Current built-ins represent idealized formal measurement information
General observed-attitude residual adapter	Planned	Built-in attitude tangents do not imply a full measured-attitude fit

See observations and schedule/multi-arc evidence.

Statistics and covariance

FEATURE	STATUS	SCOPE AND ENTRY POINT
Independent measurement noise	Supported	DiagonalNoise
Full correlated measurement noise	Supported	Exact scalar row order and mixed physical units matter; CorrelatedNoise
Correlated Gaussian priors	Supported	Named coordinates, explicit units/reference/mean; GaussianPrior
Mapped prior rows	Supported	PriorBlock; independent composition with provenance

FEATURE	STATUS	SCOPE AND ENTRY POINT
Kaula prior from shape ensembles	Supported	Separate fit/apply degrees, zonal/nonzonal laws, optional raw-moment rows; KaulaRule, KaulaPrior
Scaled joint QR/SVD covariance	Supported	Rank/null support recorded; SquareRootSolver
Local nuisance projection	Supported	Householder elimination, marginal physical support and local condition guard; NuisanceProjectedSolver
Full joint/cross covariance	Supported where numerically supported	Preserved for physical report transforms and local/shared blocks
Rank-deficient problems	Supported diagnostics	Null-sensitive uncertainty is unavailable; a pseudoinverse is not finite knowledge in null directions
Stable analysis identities	Supported	Scientific configuration/scales retained; wall-clock timing excluded from identity
Reweight saved rows	Supported	Noise/prior changes without new propagation; provider/model compatibility is checked
Formal covariance bias/truncation-error prediction	Planned	A force-order comparison conditions each covariance on its own model; it is not a bias estimate

See statistics and workflow. Priors can improve posterior support; they must not be described as improving the measurement-only information.

Independent arcs and actual estimation

FEATURE	STATUS	SCOPE AND ENTRY POINT
High-level independent-arc covariance	Supported	Shared masses/field, separate local state and channel nuisance per arc; MultiArcCovarianceStudy
Composition of frozen arc linearizations	Supported	Label/unit/physical-definition checks, shared prior once, cross-arc whitening; MultiArcStudy
Linearized residual correction	Supported	Weighted local least-squares correction and null-aware covariance; `MultiArcStudy.estimate`
Callback-based nonlinear batch/MAP driver	Partial pipeline	Implemented objective/backtracking/relinearization framework with independent recovery checks; MultiArcBatchEstimator
Ready-to-use real binary observation fit	Planned	Requires validated measured-value/residual adapters and physical context refresh/retraction callbacks
Sequential filters/smoothers	Planned	No EKF, UKF, information filter or smoother is delivered
Process noise/dynamical continuity constraints	Planned	Existing arcs are independent; no hidden process noise or continuity penalty

FEATURE	STATUS	SCOPE AND ENTRY POINT
Partial state resets/shared channel biases	Planned	High-level arc state and nuisance coordinates are local; inspect lower-level composition before custom use
Global binary state with per-arc spacecraft state	Planned	Different from the current independent binary-state formulation

The implemented generic nonlinear driver is useful infrastructure, not a claim that BASCO currently fits Hera mission data. Multi-arc documentation explains the independent-arc assumption and RK4/SO(3) restart sensitivity on chaotic trajectories.

Results, plots and reproducibility

FEATURE	STATUS	SCOPE AND ENTRY POINT
Atomic numeric artifacts	Supported	JSON/NPY, hashes, no-overwrite completion; write_artifact, read_artifact
Standard study export/reload	Supported	Descriptive run folders, tables, arrays, units and checksum verification; export_study, read_run
Shared presentation model	Supported	Numerical tables feed all formats; make_presentation
PNG/SVG/PDF and table formats	Supported	Common paper palette, grids/fonts, CSV/Markdown/LaTeX, optional external TeX; PlotTheme
User plot definitions	Supported	Replaceable figure builder consumes the canonical dataset; PlotDefinition
Hera Table 2	Supported reproduction	Four fresh default-AD cases; all 36 printed entries and all 160 report sigmas pass
Hera Figure 3	Supported reproduction	Archived cloud refits, with fresh seeded demonstration as a separate mode
Hera Figure 5	Supported reproduction	Exact degree/basis/prior cases, reused completed BASCO transports
All original R21 studies	Partial	Six same-dynamics variants checked; 13 remain unverified
Uncertainty-versus-time histories	Planned	Current fixed-window covariance output does not imply full time-dependent histories

See results, Hera, and the example gallery.

Derivative and platform evidence

FEATURE	STATUS	SCOPE
Hou AD transport	Supported, default	Optional JAX CPU float64 backend; lazy imports; no silent native fallback
Hou native analytic transport	Supported, explicit	Preserved hand-derived math/native implementation
Native-versus-AD acceptance	Supported	Same-problem tests, snapshots and Hera comparisons
Opt-in finite-difference checks	Supported selected coverage	Native RHS, parameter map, attitude retraction, high-degree/report/barycentric checks and AD/legacy checks
Complete all-chain covariance FD	Partial	Whole transport/covariance/prior companion coverage remains incomplete
Bitwise refactor snapshots	Supported on recorded host	Native 87/87 and AD 117/117 equal the Mac Phase 1 baselines after Phase 2
Managed locked macOS arm64 runtime	Validated	CPython 3.13.16; native development dependencies and current macOS deployment constraints apply
Linux portability	Partial evidence	Earlier cloud validation exists; it is not a universal release/portability certificate
Windows/release wheels for all systems	Planned/unvalidated	No supported portable-binary promise today

The Phase 2 record contains exact installed-source/runtime identities, ordinary suite outcomes and opt-in validation results. Ordinary covariance execution does not launch finite-difference probes. Derivative methods shows how to request checks deliberately.

Where future work is tracked

The authoritative scope is requirements. Implementation status distinguishes complete increments from remaining work; decisions record selected scientific/API policy. Phase 2 is complete; later modularity phases and wider real estimation/filtering scope are separate work.

CHAPTER 17

Troubleshooting and reproducibility

When something fails, identify the boundary first: environment/build, physical inputs, propagation identity, observation layout, statistical support or presentation. Fixing one boundary by quietly changing another can produce plausible but scientifically different results.

Environment and build problems

SYMPTOM	CHECK AND ACTION
Source changes appear ignored	The runner uses the installed wheel. Rebuild with `python3 -B tools/dev.py build` after library edits.
JAX dependency error	Plain studies default to AD. Prepare `ad_environment` or install the AD extra; for native-only work explicitly select `NativeAnalyticTransport()` and `run --native`.
Matplotlib unavailable	Prepare `plot_environment` and run with `--plot`; numeric exports do not need plotting.
CMake/compiler/Armadillo/OpenBLAS failure	Check the host compiler and native development dependencies. Do not repair global Python packages or install into read-only original projects.
Relative input file not found	The managed runner starts outside the checkout. Resolve script-owned paths from `Path(__file__)` or pass absolute paths.
Different machine fails	macOS arm64 is the currently validated deployment. Portability/build acceptance on other targets remains separate work.

The host Python launches the helper; the managed interpreter executes science; the wheel/native libraries provide the installed implementation. Record all three identities when diagnosing a build or run. Do not reinstall a shared runtime role while another scientific process is using it.

Physical input errors

ERROR OR SUSPICIOUS BEHAVIOR	MEANING
Missing moments	Nominal table completeness does not satisfy the chosen order. Supply them, or explicitly declare scientifically justified zeros.
Center/principal-axis rejection	First moments or products of inertia violate the supported representation. Align the geometry/moments correctly.
Unexpected scales/masses	Verify semi-axes versus diameters, density units, mass-weighted versus geometric moments and OBJ length conversion.
Invalid rotation	Supply proper mapping-direction rotations; do not silently project arbitrary invalid inputs.
Unexpected relative motion	Public velocity is inertial relative velocity expressed in primary components, not the rotating-frame derivative. Secondary spin is also expressed in primary components.

See units and frames and shape ingestion. High-degree models are especially sensitive to incomplete nominal data and incompatible normalization.

Grid and schedule errors

Cadence must be positive, duration must include at least one cadence, and the output grid must align with fixed RK4 steps. Multi-arc lengths must also align with the cadence/grid. Uniform plans start one cadence after the arc origin.

Schedules select existing epochs only. A pass window narrower than the grid can select no observations; that is a plan issue, not a reason to invent an epoch. `schedule_offset` determines mission-time phase, and `Windows` uses half-open intervals. Inspect `plan.visibility()` and `plan.schedule_description()` before propagation.

Transport reuse rejected

A retained trajectory's shape alone is not enough to establish compatibility. Physics, nominal state, parameter map/raw slots, epochs, derivative provider and integration policy enter its checked identity.

If a study was native, construct its reuse target with explicit native transport too; an AD default must not relabel native evidence. Changing gravity order or integration step requires new matching propagation. For supported statistical/channel changes, follow reuse and trades instead of editing identity fields.

Missing or surprising uncertainty

An unresolved direction is a result, not necessarily a software failure. Check:

1. Whether the duration, geometry and channel selection can identify the chosen quantities.
2. Whether initial state and barycenter nuisance are coestimated or fixed.
3. Whether raw higher-degree trace directions require explicitly stated prior knowledge.
4. Data rank versus posterior/marginal rank, coordinate scales, singular spectrum and solver policy.
5. Integration-step refinement and basis/threshold sensitivity where conditioning matters.

An unavailable full covariance cannot be replaced by arbitrary diagonals of a supported-subspace matrix. Zero nominal coefficients have no percentage uncertainty. A fixed coefficient's zero sigma means conditioned, not measured perfectly.

The projected solver's nuisance-condition error identifies a local block it cannot safely eliminate. Do not loosen its guard merely to obtain finite numbers. Reassess arc length/geometry, parameterization and the intended support policy, and preserve the rejected configuration as part of the analysis record.

Prior problems

Labels and units must match the coordinate layout. Priors with correlated quantities must retain their covariance. Shared multi-arc knowledge is supplied once globally; arc-local copies touching shared parameters are rejected.

A Kaula radius/normalization mismatch must be resolved at the physical mapping boundary, not by changing its label. For iterative fitting, maintain fixed observations, noise and prior means while rebuilding current references. Noise and priors explains the distinction.

Plotting and export problems

`tex=True` needs external LaTeX. Use `tex=False` for built-in math text when a TeX toolchain is unavailable. The plotting environment must be installed even when the covariance calculation itself succeeded.

Writers refuse an existing destination. Choose a fresh run folder instead of overwriting evidence. A failed checksum or incomplete-artifact error means the folder is not verified evidence; preserve it for diagnosis rather than bypassing verification.

`read_run` reloads arrays/reports/metadata, not a live study object. Restyling reports launches no propagation. Reanalyzing saved rows requires a correctly reconstructed scientific/layout context rather than merely loading an NPY matrix.

Minimal reproducibility record

For each result worth keeping, retain the repository commit and installed wheel/source identity; input hashes and provenance; body/state units and frames; G and reference radii; nominal gravity order and solve-for basis/body selection; epochs/cadence/gaps; derivative provider and integration step; noise/prior definitions; solver/scales/rank settings; and complete output manifests/status.

Standard exports capture much of this automatically. Add external input sources and scientific rationale through provenance. Distinguish fresh propagation, captured archive replay, reanalysis of retained transport and pure restyling.

Verify what actually completed

Do not infer completion from a directory existing or a process having started. A complete manifest, matching checksums and explicit command status are needed. Keep incomplete artifacts separate. Never overwrite frozen reference inputs or baselines when a new implementation gives a different answer.

The current project state, requirements and validation records distinguish implemented capabilities from future plans. Full binary-data fitting, custom-observable AD, all-chain FD, sequential filtering and broad portability remain open; this manual does not promote them to finished features.

CHAPTER 18

Glossary and notation

Use this page when a matrix label or physical quantity is unclear. Array shape alone does not identify its frame, units or uncertainty coordinates. The API reference supplies exact signatures; concepts connects the components.

Names and frames

TERM	MEANING IN BASCO
BASCO	Binary Asteroid Simulation, Covariance, and Observability
Primary / A	First body; Didymos in the Hera benchmark
Secondary / B	Second body; Dimorphos in the Hera benchmark
N	Inertial resolving frame
A	Primary body-fixed principal-axis frame
B	Secondary body-fixed principal-axis frame
`A_to_N` , `C_NA`	Rotation multiplying A components to obtain N components
`B_to_A` , `C_AB`	Rotation multiplying B components to obtain A components
`C_NB`	Secondary-to-inertial rotation, `C_NA @ C_AB`
DCM	Direction cosine matrix, a proper 3 × 3 rotation
Principal-axis representation	Centered moments with first moments and products of inertia structurally zero; current maintained rigid-body domain
Left attitude perturbation	`C_new = Exp(skew(delta_theta)) @ C` ; the rotation vector lives in the rotation's destination frame
SO(3)	Manifold of proper 3D rotations; attitude covariance uses local three-component tangents

BinaryState stores inertial position/velocity and both inertial spin rates **resolved in A**. Its velocity is not the time derivative of position components in the rotating A frame. Changing a resolving frame requires the appropriate rotation/kinematic transformation.

Physical symbols and units

SYMBOL / FIELD	MEANING	UNITS
`G` , `G_SI`	Fixed gravitational constant of the model	$m^3\ kg^{-1}\ s^{-2}$
`M_A` , `M_B`	Primary and secondary mass	kg
`GM` / gravitational parameter	`G × M` ; report derived with fixed G	$m^3\ s^{-2}$
`system.mass_ratio`	`M_B / M_A`	Dimensionless
`r_A` , `position`	Secondary-minus-primary inertial relative position resolved in A	m
`v_A` , `velocity`	Inertial relative velocity resolved in A	$m\ s^{-1}$
`rho_N`	Inertial relative position, `C_NA @ r_A`	m
`omega_A_A`	Primary inertial spin resolved in A	$rad\ s^{-1}$
`omega_B_A`	Secondary inertial spin resolved in A	$rad\ s^{-1}$
`theta_A_N`	Left primary attitude error resolved in N	rad
`theta_B_A`	Left relative-attitude error resolved in A	rad
`R0` , `barycenter.offset.*`	Constant barycenter position nuisance at the reference epoch	m
`V0` , `barycenter.rate.*`	Barycenter velocity nuisance	$m\ s^{-1}$
`t` , epoch	Elapsed time on the study grid	s
`dt`	Fixed RK4 integration step	s
Cadence	Time between epochs on the observation base grid	s
`R` , reference radius	Fixed normalization radius of a body	m
Bounding radius	Declared exterior-domain geometry bound; distinct from normalization radius	m

Public constructors accept the documented SI scalars/arrays or compatible Astropy quantities. Do not feed degrees into a bare angular sigma expecting an implicit degree conversion: use `np.deg2rad(...)` or an explicit `u.deg` quantity. Unit strings such as `m2` , `m3` and `kg m2` in saved layouts mean powers/products, not new parameter types.

Native30 and physical18

The retained propagator stores two DCMs explicitly. Its 30 storage values are not 30 independent physical state errors.

NATIVE SLICE	STORED QUANTITY	STORED UNITS
`0:3`	Relative position in A	km
`3:6`	Inertial relative velocity in A	km s^{-1}
`6:9`	Primary spin in A	rad s^{-1}
`9:12`	Secondary spin in A	rad s^{-1}
`12:21`	`C_NA` , flattened row-major	Dimensionless
`21:30`	`C_AB` , flattened row-major	Dimensionless

The independent physical tangent has 18 entries, in this order:

```
delta r_A [3]          m
delta v_A [3]          m/s
delta omega_A_A [3]    rad/s
delta omega_B_A [3]    rad/s
delta theta_A_N [3]    rad
delta theta_B_A [3]    rad
```

physical_lift maps physical18 errors to native30 increments. physical_dual is the left inverse; it is not simply the transpose of the lift. retract_native applies the physical error, including left rotation updates. It does not silently project or normalize the nominal state.

`BinaryState.from_native(..., project=True)` explicitly projects a propagated DCM to a nearby proper rotation before constructing a new valid initial state. High-level arcs use this to remove RK4 orthogonality drift. On chaotic tumbling that tiny restart change can grow; see multi-arc.

The state codec source is the single owner of the native layout. Standard plots decode position into SI and do not reinterpret raw state columns as meters.

Gravity, moments and inertia

TERM / SYMBOL	DEFINITION AND CAUTION
Raw inertia integral `T_lmn`	`integral x^l y^m z^n dm` , dimensional units kg m^(l+m+n)
Mass-normalized moment `That_lmn`	`T_lmn / M` , units m^(l+m+n); these are the raw-moment **solve-for coordinates** despite the shorter labels
`primary.T200`	Primary normalized `integral x² dm / M` , in m^2; not dimensional `Ixx`
Degree	`l + m + n` for a moment; spherical-harmonic degree `l` for `C_lm / S_lm`

TERM / SYMBOL	DEFINITION AND CAUTION
Mutual order	Total retained order of the Hou interaction expansion; maintained values 2, 4, 6
Nominal degree	Degree available in the body's fixed moment table
Estimated degree	Degree of independent coordinates varied in the covariance calculation; may be lower than nominal degree/gravity order
`C_lm` , `S_lm`	Exterior cosine/sine gravity harmonics; normalization and fixed reference radius must accompany values
Zonal	Harmonic order `m = 0`
Nonzonal	Harmonic order `m > 0` ; Kaula fits use a degree-wise RMS magnitude
Trace complement / trace gauge	Raw-moment directions not specified by exterior harmonics; harmonic-coordinate updates keep a declared complement fixed
Structural null direction	A parameter combination to which the given exterior data are mathematically insensitive; extra precision does not create information
Dimensional `Izz`	`M × (That_200 + That_020)` , in kg m²
Normalized polar moment	`Izz/(M R²) = (That_200 + That_020)/R²` , dimensionless

For centered principal axes, the general degree-2 report uses

$$C_{20} = \frac{2\widehat{T}_{002} - \widehat{T}_{200} - \widehat{T}_{020}}{2R^2}, \qquad C_{22} = \frac{\widehat{T}_{200} - \widehat{T}_{020}}{4R^2}.$$

These are **unnormalized** coefficients. hera_report converts to the paper's 4π geodesy normalization with no Condon–Shortley phase, and reports dimensionless polar moment. Always read a report's `conventions` before comparing another source. Varying a mass at fixed normalized shape changes dimensional moments and inertia consistently; it does not mean the shape coordinates are fixed dimensional integrals.

Priors and Kaula rules

TERM	MEANING
Prior	Explicit independent information about a named quantity, with its source/reference/units and covariance
Gaussian prior	A mean and positive covariance mapped into selected study coordinates; omitted mean uses the nominal for a covariance study
Pseudo-residual row	A whitened prior residual/Jacobian assembled alongside measurement information

TERM	MEANING
Kaula law	Degree-dependent envelope of harmonic uncertainty, here separate zonal/nonzonal fitted power laws
`K`, `alpha`	Power-law scale and slope in `K × degree^(-alpha)`
Fit degrees	Degrees used to infer a power law from an ensemble
Apply degrees	Degrees receiving prior uncertainty; a separate scientific choice
Upper envelope	Adopted bound informed by fitted scatter/interior variants; not an observed posterior
Shape ensemble	Seeded or archived perturbed geometry members with the declared coefficient normalization
Density variants	Prescribed core, shell or torus alternatives; not recovered internal-density estimates
Raw-moment prior rows	Additional independent moment information derived using the declared conversion; the paper uses these as well as harmonic rows
Fixed coordinate	Omitted solve-for; no uncertainty is being estimated for it. It is not a zero-width numerical prior

The Hera baseline constrains 36 harmonic and 56 raw-moment rows; masses and local coordinates receive no prior. The prior must be counted once when physical parameters are shared over arcs. Shapes and Kaula explains the modular producers.

Linearization, matrices and numerical support

SYMBOL / TERM	MEANING
RHS / `f`	Equations of motion, `dx/dt = f(x,p)`
State Jacobian / `A`	Derivative of the RHS with respect to the declared state coordinates; do not confuse with frame A
Parameter Jacobian / `B`	Derivative of the RHS with respect to declared parameter directions; do not confuse with body B
`Phi`, state transition	Sensitivity of the propagated state to initial state
`Psi`	Parameter sensitivity; distinguish raw-SI and independent-physical-coordinate forms
Design row / `H`	Linearized observation dependence on the full study coordinate layout
Direct parameter partial	Observation dependence on parameters outside the transported state; barycentric mass ratio is an example
Whitening	Apply the declared inverse noise factor to rows/residuals before analysis
Characteristic scale	Positive numerical conversion for mixed-unit columns; no physical prior information
QR / square root	Factorization of whitened rows without first forming ill-conditioned normal equations
SVD	Singular-value decomposition identifying numerical support/null directions

SYMBOL / TERM	MEANING
Data rank	Numerical supported dimension from measurements alone under the selected scaling/cutoff
Posterior rank	Numerical supported dimension after the stated priors are included
Marginal physical rank	Support of the physical block after local nuisance elimination
Rank cutoff / `rank_rtol`	Numerical threshold relative to the recorded spectrum; not a scientific prior
Nullspace	Directions not constrained by the analyzed information
Condition number	Ratio of relevant largest/smallest singular values; values near floating-point limits depend strongly on rounding
Nuisance projection	Eliminate local coordinates before diagnosing the marginal physical square root
Cross covariance	Off-diagonal uncertainty coupling between coordinate groups; essential in derived mass/inertia reports
Formal 1σ	Local standard deviation conditional on the nominal model, measurements, priors and numerical support policy
Relative uncertainty	`sigma / abs(nominal)` ; undefined for a zero nominal coefficient
Prior/posterior gain	Prior sigma divided by posterior sigma, for compatible reported quantities

For a supported derived quantity with Jacobian `J` , covariance transforms as `C_y = J C_x J.T` . If a requested derivative projects into an unconstrained direction, BASCO reports unavailable uncertainty rather than using a pseudoinverse to manufacture finite knowledge. Numerical rank is not automatically a physical observability count, particularly on long chaotic arcs.

The covariance solvers own the support policy; changing derivative backend should not silently change that policy. See statistics.

Derivative methods

TERM	MEANING AND CURRENT STATUS
Analytic/native	Preserved hand-derived Hou partials evaluated through compiled native code; explicitly selectable
AD	Automatic differentiation of the separate maintained Hou value equations; default JAX transport
JAX	Optional AD dependency, loaded lazily; CPU float64 is the recorded acceptance route
FD	Finite differences of independent value evaluations under physical perturbations; user-invoked validation, not routine production propagation
Retraction	Apply a physical local correction on the state manifold before a value evaluation

TERM	MEANING AND CURRENT STATUS
Known zero derivative	Explicit mathematical independence; different from missing/unsupported derivative capability
Derivative backend	Provider chosen to produce the required partials/transport; choice belongs in provenance/cache identity

AD does not remove the need to choose a physical parameterization or test equations. It differentiates the implemented value code; an incorrect value model can have perfectly consistent AD. The preserved native route and independent FD checks provide complementary evidence. See derivatives.

Arcs, estimation and results

TERM	MEANING
Arc	A time interval with its own local initial-state uncertainty
Shared parameter	One physical coordinate common to every arc, such as a body mass
Arc-local coordinate	An initial-state/nuisance coordinate independently estimated for one arc
Independent arcs	No dynamical continuity penalty or implicit process noise connects the local states
Linear correction	One weighted solution of a frozen residual/Jacobian system; does not certify nonlinear convergence
Batch/MAP iteration	Rebuild/relinearize around updated estimates while counting measurement/prior information correctly
Sequential filter	Repeated prediction/update estimator; planned, not a delivered BASCO feature
Transport	Frozen trajectory and variational sensitivities with a checked identity
LinearizedStudy	Rows, layouts, priors, observation metadata and checked transport ready for repeat analysis
CovarianceResult	Numerical covariance/support result with optional physical-report service
DerivedReport	Named values, units, Jacobian, covariance, sigma/status and conventions
Presentation	Canonical table/figure dataset built from a result; not a new dynamics calculation
Numeric artifact	Atomic JSON/NPY folder with byte hashes and completion metadata
Standard run	Study export with descriptive identity, tables/arrays/provenance and optional figures
Transport identity	Compatibility statement for model, nominal state, epoch grid, integration and provider; channels/noise can vary when compatible
Analysis identity	Scientific analysis configuration including rows, noise, prior and scales; elapsed timing does not change it
Manifest	Byte sizes and SHA256 hashes used to verify a saved folder
Snapshot	Recorded arrays/identities/tables for exact refactor comparison on the stated host

TERM	MEANING
Fresh propagation	New BASCO dynamics and variational integration
Archived-row replay	Analyze converted saved producer rows with no new dynamics; different evidence from fresh propagation
Model-conditioned trade	Compare covariance under two declared models; not an automatic estimate of modeling bias

See workflow, multi-arc, results and capability status.

REFERENCE

Public API map

Exact signatures and full method/attribute reference: API reference.

IMPORT	KIND	PURPOSE / OWNER
`G_SI`	constant	Default fixed gravitational constant, 6.674e-11 $m^3\ kg^{-1}\ s^{-2}$. Mass estimates assume this selected G is exact; pass a different fixed G to BinarySystem explicitly.
`RK4`	class	Fixed integration step in seconds, positive and finite. Both maintained Hou routes use the retained RK4 policy; this object does not select the derivative method.
`AllOf`	class	Keep epochs accepted by every component schedule (&).
`Always`	class	A schedule that retains every supplied grid epoch.
`AnyOf`	class	Keep epochs accepted by at least one component schedule (|).
`Arc`	class	One named interval of mission elapsed seconds, [start, stop].
`ArcTimeline`	class	Contiguous independent estimation arcs on one global elapsed-time timeline. A boundary observation belongs to the arc ending at that epoch. Arc boundaries must be compatible with the common grid.
`BinaryDynamics`	class	Owns nominal binary physics independently of observations, derivative provider, priors and covariance solver. Change gravity order by constructing a new model; transport reuse will reject an incompatible identity.
`BinaryState`	class	Position and inertial velocity are relative-secondary-to-primary vectors expressed in primary-body components. Velocity is not d(position_in_primary_frame)/dt. Both inertial spin vectors are also expressed in primary components. A_to_N maps primary components to inertial components; B_to_A maps secondary components to primary components. Public lengths are metres, rates radians/s, and rotation matrices dimensionless.
`BinarySystem`	class	Collect the two bodies, one initial state, fixed G and a readable system name. Initial enclosing bounding spheres must not intersect.
`Body`	class	A named UniformEllipsoid or MomentDistribution. Its mass is the raw degree-zero moment; inertia returns the three principal moments of inertia in $kg\ m^2$.
`BodyAttitude`	class	Local three-component attitude prediction errors for the selected body. sigma is in radians, not degrees; use numpy.deg2rad for degree-valued uncertainty.

IMPORT	KIND	PURPOSE / OWNER
`CoefficientEnsemble`	class	Nominal/shape/interior gravitational coefficients with body, reference radius, normalization and provenance. Ensemble coefficients are dimensionless geodesy 4π-normalized; they are not the ordinary unnormalized harmonic report.
`CoordinateLayout`	class	Named column ordering, physical units and positive numerical scales. n_state marks the leading arc-local state/nuisance block; remaining columns are shared physical parameters. Scales do not contribute prior information.
`CorrelatedNoise`	class	A positive-definite full residual covariance, in the exact row order and mixed row units of the linearization. Cholesky whitening retains correlations. Large dense matrices may be expensive.
`CovarianceResult`	class	Covariance and support diagnostics tied to an explicit named layout. A deficient supported-subspace covariance is not a full covariance. Unsupported derived uncertainty is unavailable, not zero.
`CovarianceSolver`	class	Analyze whitened SI rows and return a CovarianceResult.
`CovarianceStudy`	class	Compose nominal physics, solve-fors, observations, integration, derivative transport, noise, priors and covariance solver. Hou AD is selected when transport is omitted. Missing JAX raises an actionable error; no automatic native fallback occurs.
`DensityVariation`	class	Mass-preserving core/shell/torus interior alternatives for coefficient ensembles. Verify that chosen regions lie inside your body; the generic mesh does not prove that containment.
`DerivedReport`	class	A derived table retains labels, nominal values, units, transformation Jacobian, covariance availability and conventions. sigma is an absolute one-sigma quantity; percentages at a zero nominal coefficient are undefined.
`DiagonalNoise`	class	Positive independent residual standard deviations in each named row unit. Supplies whitening, not extra dynamics.
`DifferentiableGravityModel`	class	Optional extension capability for a JAX-traceable SI potential(r, C_AB, T_A, T_B, G). Native analytic support is not implied by supplying an AD-compatible potential.
`Every`	class	Retain epochs separated by period seconds relative to offset, when they lie on the supplied base grid. It does not interpolate or create off-grid measurements.
`GaussianPrior`	class	Named Gaussian knowledge on explicit coordinate labels/units and covariance. mean=None uses the producer reference convention; for iterative fitting preserve a fixed intended mean rather than recentering it at every iteration.
`GravityModel`	class	Runtime-checkable minimal gravity declaration with mutual_order. Member-presence checks do not establish backend compatibility or physical correctness; maintained Hou evaluators own the Hou equations.
`HouGravity`	class	The maintained binary gravity workflow uses total mutual expansion order 2, 4 or 6. This nominal order is separate from the estimated moment/harmonic degrees, which currently run through degree 4.

IMPORT	KIND	PURPOSE / OWNER
`IndependentPriors`	class	Compose independent prior producers while preserving each block metadata and its internal correlations. Independence between blocks is an assumption, not inferred from their names.
`IntegrationSchedule`	class	Provider-level propagation/output policy, independent of observation weights and priors. Epochs are increasing positive elapsed seconds; transports also retain the epoch-zero initial state.
`KaulaPrior`	class	Map the rule normalization/radius through the physical-coordinate Jacobian. raw_moment_priors=True additionally selects the paper's independent RMS-touching raw-moment constraints; it is separate from harmonic knowledge and adds no implicit mass/state/barycenter prior.
`KaulaRule`	class	Fitted zonal/nonzonal normalized-coefficient magnitude laws and adopted sigmas. Persist with to_dict/from_dict. A law describes an explicit empirical prior assumption rather than an observed covariance result.
`LeastSquaresSolver`	class	Optional correction on the same numerical support as its covariance.
`LinearizedStudy`	class	Owned, frozen prediction rows with named row/column layouts, nominal, transport, noise/prior context and content identity. Reanalysis changes statistical assumptions without rerunning dynamics.
`MasconSet`	class	Point masses fixed in a body frame: positions in metres, masses in kilograms.
`MomentDistribution`	class	Explicit centered principal-axis mass moments in SI. The reference radius fixes harmonic normalization; the bounding radius supports non-overlap checks; characteristic_length sets numerical coordinate scales. Missing moments are errors unless missing="zero" is deliberately selected.
`MultiArcBatchEstimator`	class	Generic callback-based nonlinear batch/MAP loop with relinearization and objective backtracking. Callers supply correctly signed residuals and physical retractions. Mission-style binary observed-value adapters, recovery validation and multiple filters remain future work.
`MultiArcBatchResult`	class	Returned state, study, covariance, status and iteration history from the generic batch loop. Inspect explicit convergence status; a returned result does not automatically imply convergence.
`MultiArcCovarianceStudy`	class	High-level binary multi-arc covariance: one continuous nominal, independent local initial-state errors per arc, shared masses/gravity, and optional arc-local nuisance. Shared priors enter once. There is no implicit continuity constraint.
`MultiArcEstimate`	class	A single weighted linearized correction, with null-aware covariance.
`MultiArcStudy`	class	Assemble arbitrary already-linearized arcs into named local columns plus shared physical columns. Default measurement noise is independent across arcs; a full joint whitening model can express cross-arc correlation.
`NativeAnalyticTransport`	class	Explicit preserved Hou values plus hand-derived partials. Supports the centered principal-axis order-2/4/6 Hou workflow and fixed RK4. This route does not require JAX.
`NoiseModel`	class	Whiten a vector or row matrix without changing its shape or ordering.
`Not`	class	Invert a component schedule (~).

IMPORT	KIND	PURPOSE / OWNER
`NuisanceProjectedSolver`	class	Alternative Householder elimination of local nuisance directions with marginal physical support diagnostics. Inspect joint versus marginal rank; its support criteria need not produce identical availability decisions to joint scaling.
`ObservationChannel`	class	Prediction-direction contract: named labels/units, positive noise sigma, and signed local native-state/direct-parameter partials. Actual fixed-data residuals for nonlinear fitting are a separate contract.
`ObservationPlan`	class	Owns one regular base grid and channel definitions. Schedules remove selected observation rows on this grid; they do not skip propagation stages. Arbitrary off-grid observation epochs are not implemented by the fixed-step transport.
`ObservationSchedule`	class	Select epochs on the plan's uniform grid without changing dynamics.
`Parameters`	class	A readable solve-for builder attached to one exact nominal model object. Choose independent masses, moments or higher-degree exterior harmonics, then optionally estimate the 18 physical initial-state errors. Builder call order determines column order.
`Periodic`	class	A periodic keep-window schedule on the existing epoch grid. period, duration and phase are in elapsed seconds.
`PeriodicBlackout`	class	Remove a repeated blackout duration within a period, in seconds. Combine masks with &, | and ~; consult the source for boundary conventions.
`PlotDefinition`	class	A replaceable named figure drawing function and caption. Custom figures consume canonical presentation data rather than recomputing estimation results.
`PlotTheme`	class	Shared palette, typography, grids and output formats. tex=False uses Matplotlib mathtext without an external TeX install. tex=True requires a working TeX environment; use the same theme across comparison figures.
`PolynomialBias`	class	Observation-only polynomial coordinates with explicit units/reference epoch. Degree 1 and three components add barycenter position and rate; no prior is introduced implicitly. These are nuisance parameters, not a change to nominal binary gravity.
`PrimaryBarycentricPosition`	class	Predict the primary position relative to system center of mass: -M_B/(M_A+M_B) times inertial relative separation, plus optional bias. Includes direct mass-ratio derivatives as well as transported dynamical effects. sigma is in metres.
`PriorBlock`	class	Advanced mapped prior rows with explicit layout, mean/reference and noise. Supports a prior on a derived linear combination without pretending it is a coordinate-wise independent constraint.
`PriorProvider`	class	Produce a concrete basco.PriorBlock, without selecting a solver.
`RadialShapeSampler`	class	Seeded radial shape perturbations with requested RMS roughness and correlation lengths in metres. Choose sample count, resolution, frame/realignment and legacy seed convention explicitly for a science campaign.

IMPORT	KIND	PURPOSE / OWNER
`RelativePosition`	class	Three relative-position prediction directions, with sigma in metres. Only frame="inertial" is currently implemented.
`ResolvedParameters`	class	The physical-coordinate map owns labels/units and derived physical reports, while numerical covariance solvers remain independent of binary physics. raw_jacobian includes mass-to-moment pathways, not just selected diagonal entries.
`Scheduled`	class	Wrap one existing channel to give it its own cadence, pass windows or blackouts on the common plan grid. The wrapper preserves the channel labels, units and native partial method.
`SquareRootSolver`	class	Default joint scaled solver using SVD for numerical support and QR for covariance/correction. Positive scales and rank_rtol control numerical representation/support; neither adds information.
`Transport`	class	Immutable epochs, native30 states, state transition, raw SI-moment sensitivities, physical-coordinate sensitivities, diagnostics and validity identity. Save labels/units and identity with numerical arrays; raw and physical sensitivity axes are different.
`TransportIdentity`	class	A canonical content identity over declared physical/numerical inputs and derivative-coordinate mappings. It prevents accidentally reusing a transport for changed gravity or solve-for derivatives; it is not proof of numerical accuracy.
`TransportProvider`	class	A replaceable provider owns physics propagation and variational transport; it must return the owned Transport with correct layouts, units and content identity. It does not choose observations, priors or covariance solver.
`TriangleMesh`	class	A closed, outward-wound triangular surface in metres with provenance. Topology/volume checks do not certify absence of every possible self-intersection or interior-region containment.
`UniformEllipsoid`	class	A homogeneous ellipsoid with positive semi-axes in metres and density in kg/m³. Supplies analytic mass-weighted moments through degree 6; estimating independent moments does not constrain the uncertain body to remain homogeneous.
`VariationalEvaluator`	class	Callable value-plus-derivative evaluation F, A=dF/dx and B=dF/dT used by the common variational integration owner. No specific force implementation is required by this structural protocol.
`Windows`	class	Keep epochs inside explicit elapsed-second intervals; Windows.around builds windows around listed pass centers.
`analyze_design`	function	Analyze already-whitened named-layout rows directly. If you start from physical measurement partials, apply the correct noise whitening first; passing raw rows changes the statistical problem.
`build_shape_ensemble`	function	Generate coefficient statistics from shape and optional density members, independently of dynamical order. realign=True aligns each perturbed copy; False retains its perturbation-frame convention. These are scientifically different ensembles.

IMPORT	KIND	PURPOSE / OWNER
export_study	function	Write the standard descriptive run folder with configuration, named arrays, presentation, provenance and checksums. Its analysis identity excludes runtime timing while retaining science/statistical choices and scales.
make_presentation	function	Build one canonical result dataset that feeds all tables and plots. Supply rows to retain linearization provenance; custom derived reports can join the same presentation.
physical_dual	function	A checked left inverse of physical_lift. Transposing the lift alone does not provide its dual.
physical_lift	function	Map an 18-component SI/local-attitude error into native30 tangent space. Left attitude perturbations are used; the first six components include metre-to-kilometre conversion.
read_artifact	function	Verify schema, array shapes/dtypes and hashes before returning arrays/metadata. Untrusted paths and external symlink targets are rejected by shared readers.
read_presentation	function	Verify and reload canonical table/plot input without rerunning dynamics. It does not reconstruct a live study.
read_run	function	Verify and reload the standard run into arrays, presentation and run metadata. Numeric evidence is reloaded, not a live model/study.
references	module	basco.references
retract_native	function	Apply a finite physical SI/local-SO(3) error to a native30 state. This is an explicit estimation/retraction utility rather than a propagation-time correction.
run_path	function	Descriptive UTC name plus identity; never overwrite an existing run.
schedules	module	basco.schedules
transport_identity	function	Identify physical inputs, derivative coordinates and numerical policy.
write_artifact	function	Atomic JSON/NPY evidence container with hashes and shape/dtype declarations. Object arrays and pickle are excluded; preserve named unit/layout metadata explicitly.
write_presentation	function	Atomically write canonical JSON, Markdown, CSV and LaTeX tables, plus optional PNG/SVG/PDF figures. The destination must be new; existing completed results are not overwritten.
write_run	function	Atomic complete run: overview, config, reports, arrays and validation.